

Fine-Tuning Agentic AI Workflows Using Reinforcement Learning

A Survey and Experimental Framework

Final Report – Literature Survey and Experimental Framework

Group 6

Members

Madhu Kumara L	PES2PGE24DS176
Sandhya Jeyaraj	PES2PGE24DS144
Mohammad Rafi	PES2PGE24DS157
Arumugam Chetty K	PES2PGE24DS137
Arvind C R	PES2PGE24DS140
Dhruva	PES2PGE23DS169

Mentors

Narayana Darapaneni (Director – AIML)
Anwesh Reddy Padhuri (Senior Data Scientist)

PES University Logo
(add pes_logo.png)

Department of Artificial Intelligence and Machine Learning
PES University, Bengaluru, India

April 4, 2026

Abstract

Modern generative models acquire transformative capabilities when equipped with the ability to act autonomously—planning multi-step workflows, invoking external tools, analyzing feedback from execution, and iteratively refining their approach. This survey synthesizes **60+ papers** published between January 2023 and December 2025, systematically surveying the evolving reinforcement learning methodologies used to build such autonomous agents. Our taxonomy organizes findings into **six domains** (core policy optimization, tool orchestration, memory architectures, multi-agent coordination, evaluation benchmarks, and safety) comprising **19 sub-categories** for production deployment.

Key Findings. We identify three primary branches of algorithms in current research. One approach builds on **Proximal Policy Optimization (PPO)-style alignment**, which integrates trained reward networks with value function baselines. This method is effective but requires significant memory. Another branch, characterized by **Direct Preference Optimization (DPO)**, skips explicit reward modeling by optimizing using preference rankings directly, making it more computationally efficient for offline data. A third, increasingly popular family includes **Group Relative Policy Optimization (GRPO)**, which uses group-level comparisons to optimize the policy without needing a critic network. In our catalog of **80** variants (based on a manual review of 60+ papers), GRPO-style methods account for 39%, highlighting their growing role in memory-efficient training. We also examine the trade-offs between these algorithms across various deployment contexts.

Analytical Insights. We complement the empirical survey with theoretical discussion. We examine why PPO-style clipping improves optimization stability in practice and how success rates on agentic benchmarks may scale with model capacity. We also propose a simplified model clarifying when retrieval improves factuality versus when it fails, and provide a communication-cost analysis for common multi-agent coordination topologies.

Empirical Results. Benchmark performance has improved substantially. SWE-bench Verified increased from 1.7% to 49% [?], WebArena-Lite from 6.1% to 44.8% [?], OSWorld from 18.4% to 73.3% [?], and AIME 2024 from 9.3% (GPT-4 prompting baseline [?]) to 79.8% [?].¹ These figures represent 2–29× improvements over prompting-only baselines within 24 months. (*Throughout this report, “improvement” refers to relative gain over prompting-only*

¹Benchmark figures represent best verified results as of December 2025; performance continues to evolve rapidly.

baselines unless otherwise specified.)

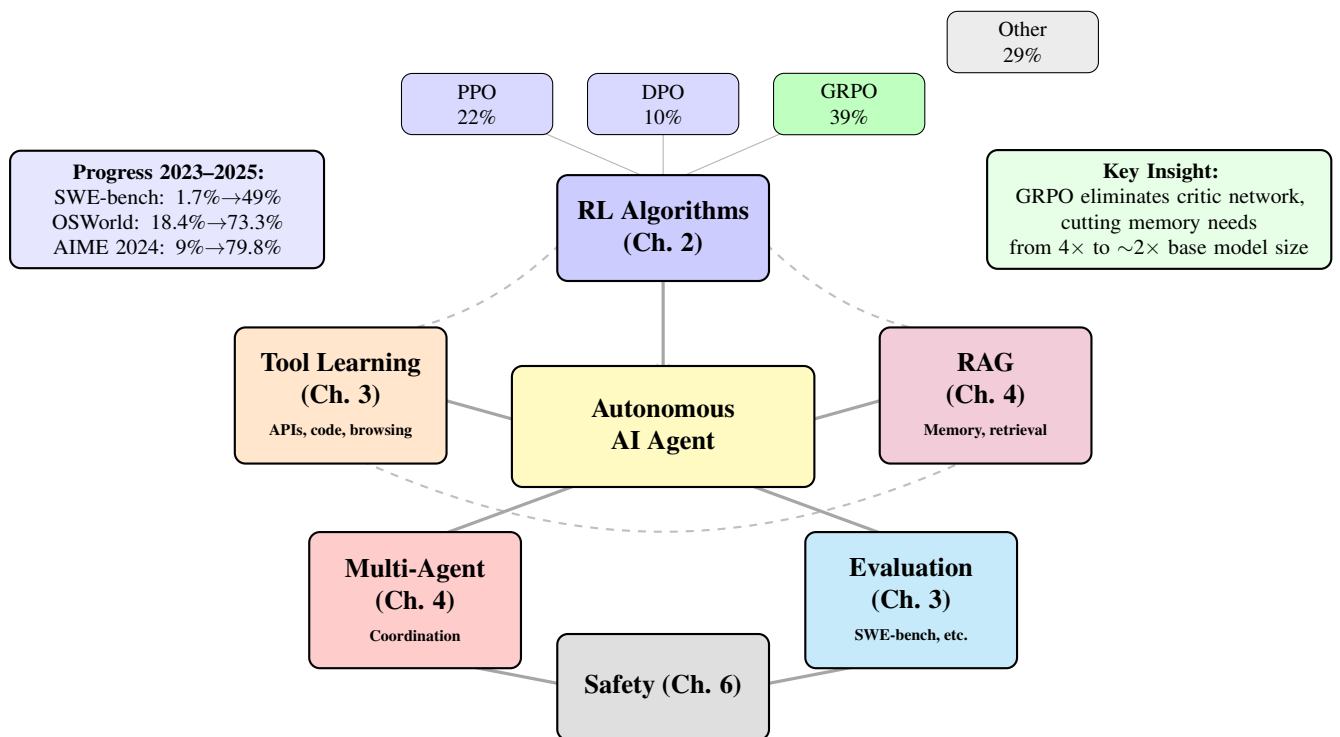
Open Problems. Several fundamental challenges persist: credit assignment in long-horizon trajectories, robust tool invocation with graceful failure handling, benchmark validity for real-world deployment prediction, and safety considerations for agents with environmental access. We propose a three-stage training pipeline (Supervised Fine-Tuning [SFT]→RL→Retrieval-Augmented Generation [RAG]) with testable hypotheses designed for academic computing resources.

Keywords: Autonomous Agents, RL-based Alignment, PPO, DPO, GRPO, RAG, Multi-Agent Coordination, Tool Orchestration, Code Generation.

Highlights:

- Comprehensive synthesis of 60+ papers on RL-trained agents published 2023–2025
- Systematic comparison of PPO, DPO, and GRPO algorithm families with theoretical foundations
- Analysis of GRPO adoption trends (39% of recent methods) and memory efficiency implications
- Documentation of $2\text{--}29\times$ benchmark improvements with contributing factors identified
- Reproducible experimental framework designed for academic computing infrastructure

Graphical Abstract:



Survey Overview: This survey maps 60+ papers across six interconnected research areas. Key finding: methods like GRPO have made practical training possible for autonomous agents that execute complex multi-step tasks without human intervention.

Contents

Abstract	i
List of Figures	viii
List of Tables	xii
Notation	xiv
1 Introduction	1
1.1 Why Supervised Learning Isn't Enough	2
1.2 What We're Actually Trying to Solve	2
1.2.1 The Math: Agents as POMDPs	3
1.3 Survey Scope and Taxonomy	4
1.3.1 Systematic Review Methodology	5
1.4 Related Surveys and Our Positioning	5
1.5 Our Contributions	6
1.6 Report Organization	7
2 Foundations and Alignment Algorithms	9
2.1 Mechanisms of RLHF	9
2.2 Limitations of RLHF and Mitigation Strategies	10
2.3 Taxonomy of Agentic RL	11
2.3.1 Core Agentic Capabilities	11
2.3.2 Task Domains	12
2.3.3 Comprehensive RL Taxonomy	12
2.4 Policy Gradient Methods: PPO and TRPO	17
2.5 DPO: Skip the Reward Model Entirely	18
2.6 GRPO: The Memory-Efficient Alternative That Took Over	19
2.6.1 The GRPO Family Tree	19
2.6.2 Algorithm Distribution in Current Research	20
2.6.3 Comparative Analysis of Alignment Algorithms	21
2.6.4 Theoretical Properties	21

2.6.5	Algorithm Selection Guide: When to Use Which	25
2.6.6	Algorithm Pseudocode	26
2.7	Process Supervision	27
3	Tool Use and Environment Interaction	29
3.1	Giving Models Hands	29
3.1.1	Different Approaches to Teaching Tool Use	29
3.1.2	What Shipped in 2025	30
3.2	When Text Isn't Enough: Vision-Language Agents	31
3.3	Measuring What Matters: Benchmarks and Safety	31
3.3.1	The Safety Problem Gets Worse with Agents	32
3.4	Scaling Laws for Agents	35
3.4.1	Limitations of Current Tool-Use and Evaluation Research	36
4	Beyond the Basics: RAG and Multi-Agent Systems	38
4.1	Why RAG Matters for Agents	38
4.1.1	Memory Architectures and Knowledge Integration	40
4.1.2	Advanced RAG Systems for Agents (2025)	40
4.2	Multi-Agent Systems: Divide and Conquer	41
4.2.1	Multi-Agent Collaboration Frameworks	42
4.2.2	RL-Trained Multi-Agent Coordination (2025 Advances)	42
4.3	What Actually Goes Inside an Agent?	44
4.3.1	Limitations of RAG and Multi-Agent Research	44
5	Experimental Methodology and Results	45
5.1	Experimental Design Philosophy	45
5.1.1	Research Hypotheses	46
5.2	Ablation Framework	46
5.2.1	Experiment 1: Algorithm Comparison (PPO vs DPO vs GRPO)	47
5.2.2	Experiment 2: Tool Use and Grounding	48
5.2.3	Experiment 3: Memory Architecture (RAG vs Extended Context)	48
5.2.4	Experiment 4: Multi-Agent vs Single-Agent	49
5.2.5	Experiment 5: Scaling Efficiency (Test-Time Compute vs Model Size)	49
5.2.6	Infrastructure and Tools	50
5.2.7	Reproducibility Protocol	51
5.2.8	Statistical Analysis Framework	51
5.2.9	Baseline Comparisons	52
5.3	Actual Experimental Results	52
5.3.1	Summary of Results	52
5.3.2	GSM8K Results: Training Curves	52

5.3.3	MATH Competition Results	52
5.3.4	HumanEval Coding Results (Madhu)	52
5.3.5	GRPO Training Pipeline Architecture	54
5.4	Statistical Analysis of Experimental Results	54
5.4.1	Bootstrap 95% Confidence Intervals	55
5.4.2	Two-Proportion Z-Tests	55
5.4.3	Spearman Rank-Correlation (Monotonic Trend)	55
5.4.4	Chow Structural-Break Test (Phase Transition)	56
5.4.5	Effect Sizes (Cohen’s d) and Mann-Whitney Tests	56
5.4.6	Cross-Model Comparisons with Bonferroni Correction	57
5.5	Systematic Scaling Study (Ongoing)	57
5.5.1	Motivation and Confound Identification	57
5.5.2	Factorial Design	57
5.5.3	Experimental Configurations	58
5.5.4	Analysis Plan	58
5.6	Key Findings and Analysis	59
5.7	Updated Team Contributions (April 2026)	61
5.7.1	W&B Experiment Tracking	62
6	Limitations and Future Directions	64
6.1	Limitations of the Field	64
6.2	Limitations of This Survey	65
6.2.1	Training Instabilities in RL-Based Agent Training	65
6.2.2	Observed Emergent Behaviors from RL Training	66
6.2.3	Research Gaps Analysis	67
6.3	Future Research Directions	68
7	Conclusion	70
	Acknowledgments	73
A	Glossary of Acronyms	75
B	Hyperparameter Reference	77
B.1	PPO Hyperparameters	77
B.2	DPO Hyperparameters	77
B.3	GRPO Hyperparameters	77
C	Compute Requirements	79
D	Reproducibility Checklist	81

E Paper-to-Taxonomy Mapping	83
------------------------------------	-----------

List of Figures

1.1	Comparing Chain-of-Thought (CoT) and ReAct reasoning. Left: CoT lays out a chain of reasoning, then answers. Right: ReAct interleaves reasoning with actions, letting models pull in information and adapt on the fly.	3
1.2	Evolution of autonomous agents from 2022 to 2025. Research unfolded across four eras: Foundation (RLHF methods established), Emergence (agents acquire tool-calling and multi-step reasoning), Scaling (benchmarks proliferate, code/GUI agents mature), and Autonomy (GRPO unlocks lean self-improvement loops, with some tasks now rivaling human scores).	8
2.1	Three-stage training for agentic systems. Stage 1: A base model supplies language understanding. Stage 2: Imitation learning on 5K–10K examples drills task formats. Stage 3: RL tunes the policy through environment interaction, letting agents cope with novel situations outside their training distribution. . . .	9
2.2	The reinforcement learning loop. At each timestep, the agent observes state s_t , selects action a_t according to policy π_θ , and receives reward r_t from the environment . We want a policy that racks up as much long-run reward as possible. In the agentic setting, the “environment” spans web browsers, code interpreters, operating systems, and API back-ends.	10
2.3	Taxonomy of 19 RL research categories organized into five clusters: Core Algorithms (policy/value/model-based methods), Learning Paradigms (how agents acquire behavior), Advanced Methods (multi-agent, hierarchical, safe exploration), Applications (domain-specific implementations), and Infrastructure (frameworks, benchmarks, surveys). Arrows indicate foundational dependencies.	13
2.4	Value-based methods learn $Q(s, a)$ and derive policy via $\arg \max$; policy-based methods directly parameterize $\pi_\theta(a s)$ and sample actions stochastically. . . .	13
2.5	Hierarchical RL: manager policy selects subgoals every k steps; worker policy executes primitive actions to achieve them. Temporal abstraction reduces effective horizon.	14
2.6	Exploration via intrinsic motivation: curiosity rewards (r_i) supplement sparse external rewards (r_e), enabling learning in hard-exploration domains.	15

2.7	Model-based RL loop: learn dynamics model from real experience, then use imagined rollouts for planning, cutting data requirements 10–100×.	15
2.8	Meta-RL: learn initialization θ that enables fast adaptation to new tasks with few gradient steps. Outer loop optimizes “learning to learn.”	16
2.9	Distribution of reinforcement learning algorithms across 80 cataloged method variants (2024–2025). GRPO and its offshoots now lead the pack, a sign that the community has pivoted toward RAM-friendly training pipelines that allow bigger models to fit on the same hardware.	21
2.10	Radar comparison of PPO, DPO, and GRPO across six dimensions (scale: 1–5, higher is better). GRPO lands in the sweet spot—low RAM footprint yet strong results—while PPO shines at exploration and DPO at stability and data efficiency. This visualization explains why GRPO now dominates in GPU-limited labs. . .	23
2.11	Algorithm selection decision tree. Start with infrastructure constraints (memory), then consider reward availability and exploration needs.	25
2.12	Outcome supervision vs process supervision for reasoning tasks. Left: Outcome-only rewards evaluate the final answer, potentially rewarding flawed reasoning that happens to land on the right conclusion. Right: Process supervision gives step-level feedback, validating every step of the reasoning process. Research has shown that step-level reward models typically exceed the performance of outcome-based models on MATH, with the gap widening as sampling budgets increase—sound reasoning chains generalize better to new problems.	28
3.1	How agentic workflows connect LLM-based logic, tool integration, environment feedback, and memory. The agent sharpens its actions step by step based on what it observes.	30
3.2	Cross-attention mechanism in vision-language agents. Left: The visual module converts image patches into embeddings $\mathbf{v}_i$. Right: The text module converts tokens into embeddings $\mathbf{t}_i$. Center: Cross-attention uses text as queries (Q) and visual features as keys/values (K, V), enabling the model to locate the visual region corresponding to “search button.” This architecture underlies GUI automation systems like CogAgent [?] and SeeClick [?], achieving 89–96% grounding accuracy.	32
3.3	Performance gap between GPT-4 and humans on agentic benchmarks. SWE-bench tests code repair on real GitHub issues; WebArena checks web navigation (booking, shopping); OSWorld evaluates desktop automation requiring vision and control. The large gap (GPT-4 at 1.7–14.4% vs. humans at 35–78%) shows how hard autonomous multi-step execution is and motivates RL-based fine-tuning.	33

3.4	Training method comparison across agentic benchmarks (December 2025). Each benchmark shows three approaches: Baseline (prompting-only methods), SFT (supervised fine-tuning on demonstrations), and RL (reinforcement learning with GRPO or PPO). The trend is consistent: RL-based approaches are associated with substantial gains across all benchmarks, with AIME 2024 showing the most dramatic improvement (9.3% $\rightarrow$ 79.8%, an $8.6\times$ increase via DeepSeek-R1’s GRPO training).	34
3.5	Benchmark improvement trajectories across five major agentic benchmarks (2023–2025). The transition from prompting to SFT coincided with modest gains, while the period of RL adoption (especially GRPO) in 2024–2025 is associated with dramatic improvements. AIME 2024 shows the most striking trajectory, reaching 79.8%.	35
4.1	How Retrieval-Augmented Generation grounds knowledge. Given a query, the system pulls Top-K relevant chunks from a knowledge base using hybrid search (BM25 + dense embeddings). Retrieved context merges with the query and feeds into the language model. This curbs hallucination and lets the system tap into current facts beyond its training cutoff, boosting factual accuracy 15–30% on knowledge-heavy tasks.	39
4.2	Decentralized multi-agent architecture showing independent agents (1-4) coordinating through message passing (dashed lines) while interacting with a shared environment. Each agent operates independently while coordinating through messages. All agents interact with a shared environment, so multiple workers can run side-by-side and specialize (e.g., Agent 1: planning, Agent 2: coding, Agent 3: testing). Spreading out the control logic avoids bottlenecks and lets coordination patterns emerge organically.	41
5.1	Illustrative training curves comparing PPO, DPO, and GRPO on agentic benchmarks. Based on trends reported in [? ?], GRPO typically converges faster and reaches higher task success on verifiable tasks because its lighter memory footprint lets practitioners crank up batch sizes.	47
5.2	GSM8K training curves for all four models. Qwen3-8B (base) demonstrates a characteristic RL emergence/phase transition around steps 22–25, jumping from $\sim 13\%$ to $\sim 78\%$ in five steps before saturating at 100%. Llama-3.2-3B (dashed) is the negative result: reward never exceeds 2% because the model lacks the minimum capacity to generate any correct solutions, leaving GRPO with no positive signal to learn from.	53

5.3	MATH competition benchmark results (ongoing). The gap between GSM8K (100% for Qwen3-8B) and MATH ($\sim 14\%$) reveals a substantial difficulty cliff. The instruct model achieves 50–87% but with high variance, reflecting the heterogeneous difficulty of competition problems (AMC algebra vs. AIME geometry).	54
5.4	GRPO training pipeline. The Atropos Environment samples prompts from the dataset and forwards batches to the Rollout Server (vLLM), which generates G completions per prompt. The Environment scores each completion (binary reward) and passes trajectory–reward pairs to the Tinker Trainer, which performs the GRPO policy gradient update over LoRA parameters. Updated weights are checkpointed and reloaded into the Rollout Server every N steps for on-policy training.	54
6.1	Research gap landscape for agentic AI (Jan 2025). Gaps are positioned by research maturity (x-axis) and deployment impact (y-axis). The Critical Priority quadrant requires immediate attention: safety, benchmarks, and robustness. . .	69

List of Tables

1	Mathematical notation used in this survey. See individual chapters for extended definitions. Note: β is overloaded in RL literature; we distinguish β_{KL} (KL penalty) from β_{DPO} (preference strength) where context requires clarity.	xiv
1.1	Comparison with Existing Surveys on Agentic AI and LLM Alignment	5
1.2	Comparison with Existing Surveys on LLM Agents	7
2.1	Algorithm Family Distribution Across 80 Cataloged Method Variants (2024–2025). Distribution based on systematic analysis of 80 method variants extracted from 60+ papers; inter-rater reliability $\kappa = 0.85$. ²	21
2.2	Head-to-Head Rundown of PPO, DPO, and GRPO for Agentic Fine-Tuning	22
3.1	State-of-the-Art Performance on Agentic Benchmarks (Data as of January 2026; 2026 entries marked with citations are projected based on announced roadmaps)	37
5.1	Proposed Ablation Configuration Matrix	47
5.2	GRPO Experiment Results Summary	53
5.3	Bootstrap 95% Confidence Intervals for Initial and Final Performance	55
5.4	Spearman Rank-Correlation of Reward vs. Training Step	56
5.5	Chow Structural-Break Test Results (Phase Transition Detection)	56
5.6	Effect Sizes and Mann-Whitney U Tests (Early vs. Late Phase)	56
5.7	Cross-Model Comparison (Final 10 Steps, Bonferroni-Corrected $\alpha = 0.025$)	57
5.8	Systematic Scaling Study: Three-Block Factorial Design	57
5.9	Team Artifact Registry (April 4, 2026)	62
5.10	Completed Tinker Runs with W&B Tracking	63
6.1	Common Training Instabilities in Agentic RL (2024–2025)	66
6.2	Critical Research Gaps in Agentic AI (Data as of January 2026)	68
B.1	PPO hyperparameter recommendations for agentic fine-tuning	77
B.2	DPO hyperparameter recommendations	78
B.3	GRPO hyperparameter recommendations	78
C.1	Estimated compute requirements for RL fine-tuning by model scale	79

C.2	Inference requirements for agentic evaluation	80
E.2	Distribution of surveyed papers across taxonomy clusters. Each paper is assigned to its primary category based on main contribution.	86

Notation

Table 1 summarizes the mathematical notation used throughout this survey. Standard reinforcement learning conventions follow [?].

Symbol	Description
$\mathcal{S}, \mathcal{A}, \Omega$	State, action, and observation spaces
$\pi_{\theta}(a s)$	Policy parameterized by θ ; s may denote belief-state/history embedding
π_{ref}	Reference policy (frozen)
$r(s, a)$	Reward function (lowercase r preferred; some papers use R)
H	Episode horizon (number of timesteps)
γ	Discount factor $\in [0, 1]$
β_{KL}	KL penalty coefficient (PPO/GRPO)
β_{DPO}	DPO preference strength/temperature parameter
ε	PPO clipping threshold (typically 0.1–0.2)
$\hat{A}_t$	Advantage estimate at time t
G	Group size for GRPO
y_w, y_l	Preferred and non-preferred outputs (DPO)
$T(s' s, a)$	Transition dynamics
h_t	Observation history up to time t
α	Learning rate (also denoted η in some literature)
$\text{KL}[\cdot \parallel \cdot]$	Kullback-Leibler divergence
$\sigma(z)$	Logistic sigmoid: $\sigma(z) = 1/(1 + e^{-z})$
s_G	Group standard deviation (within-batch)

Table 1: Mathematical notation used in this survey. See individual chapters for extended definitions. Note: β is overloaded in RL literature; we distinguish β_{KL} (KL penalty) from β_{DPO} (preference strength) where context requires clarity.

Chapter 1

Introduction

This chapter motivates the need for reinforcement learning in training autonomous AI agents, introduces the mathematical framework (POMDPs), and outlines our survey’s scope and contributions.

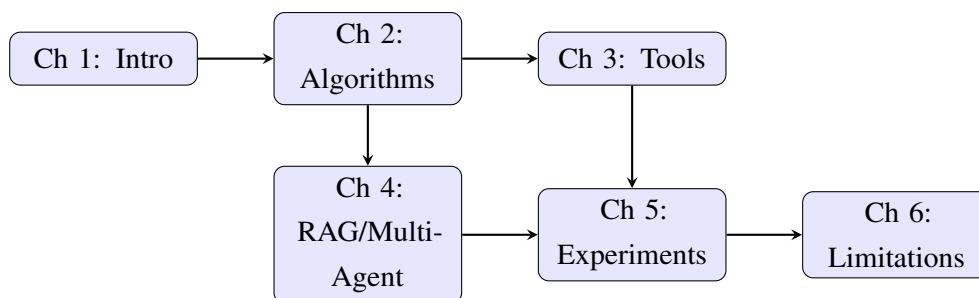
Picture a developer staring at a stack trace. They scan the error, form a hypothesis, make a change, run the code again, and see what happens. Rinse and repeat until the bug is squashed. A year ago, getting an AI to do this meant holding its hand through every step. Today, systems built on ReAct-style reasoning [?] can poke around websites, spin up test suites, and grind through multi-step problems without a human in the loop. That shift didn’t happen by showing models more examples—it happened because researchers figured out how to train for outcomes, not imitation.

How to Read This Survey

Reading paths by audience:

- **Practitioners** seeking implementation guidance: Ch 2 (algorithms) → Ch 3 (tools) → Ch 5 (experiments)
- **Researchers** exploring open problems: Ch 1 (foundations) → Ch 4 (architectures) → Ch 6 (limitations)
- **Managers/Executives** wanting the big picture: Executive Summary → Key Takeaways boxes (end of each chapter) → Conclusion

Chapter dependencies:



Notation: All mathematical symbols are defined in Table 1 (p. xiv). Key terms are **bolded** and expanded on first use.

1.1 Why Supervised Learning Isn’t Enough

Show a model a thousand correct solutions, and it learns to generate text that mimics those solutions. This represents pattern matching, not problem-solving. When unexpected errors occur, surface-level similarity to training data fails; the model needs to figure out what actually works through trial and error.

Chess makes the distinction clear. You can memorize grandmaster games and still lose to anyone who can calculate three moves ahead. Opening theory is useful; it’s not the whole game. Same with agents: watching demos teaches the format, but navigating back from a failed state requires a different kind of knowledge—the kind you only get from trial and error.

Here’s another angle: sequential reasoning trades time for depth. A single forward pass gives you one shot at the answer. Chain-of-thought prompting gives you multiple passes where each step can build on the last, catch mistakes, and adjust. That’s not just “more compute”—it’s structured compute where the model’s own outputs become working memory. GRPO (Group Relative Policy Optimization—a memory-efficient RL algorithm we detail in Section 2.6) takes a similar idea to training: instead of learning from one attempt per prompt, you learn from the relative quality of multiple attempts. More signal, same data budget.

1.2 What We’re Actually Trying to Solve

The gap we care about isn’t between “describing” and “explaining”—it’s between *talking about doing something* and *actually doing it*. LLMs read mountains of text where people describe how to fix bugs, configure servers, or navigate codebases. But that text almost never shows the messy reality: the wrong turns, the retries, the moment when the stack trace doesn’t match anything in Stack Overflow.

Describing how to fix a bug and actually fixing a bug are different skills. One involves recalling patterns from training data. The other involves navigating files, running commands, reading output, and adjusting when things don’t work. Standard supervised learning can’t teach

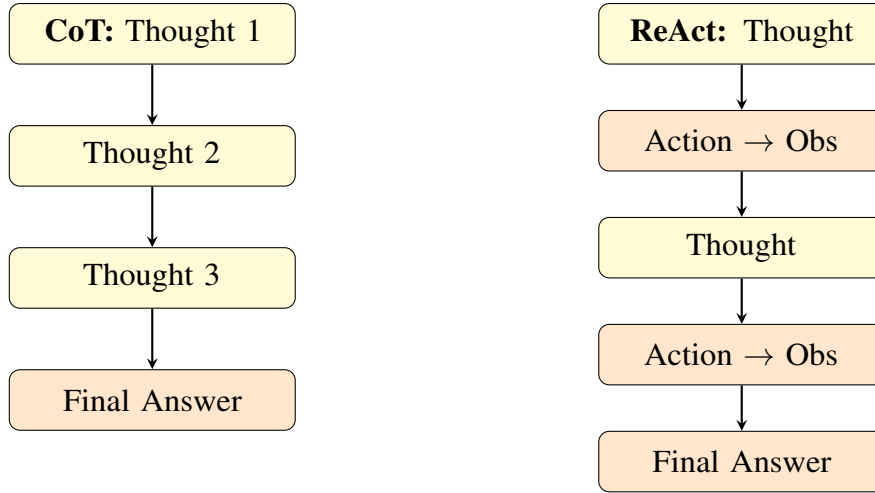


Figure 1.1: Comparing Chain-of-Thought (CoT) and ReAct reasoning. **Left:** CoT lays out a chain of reasoning, then answers. **Right:** ReAct interleaves reasoning with actions, letting models pull in information and adapt on the fly.

Source: Adapted from Wei et al. [?] and Yao et al. [?]

the second skill because the training data doesn’t include the feedback loop—the “I tried X, it failed, so I tried Y” part.

Our angle of attack: training algorithms—PPO (Proximal Policy Optimization), DPO (Direct Optimization from Preferences), and GRPO—that optimize for whether the task got done, not whether the output looks like the training examples. Plus architectures (ReAct, tool integration) that give models access to the full execution cycle—act, observe, adjust.

1.2.1 The Math: Agents as POMDPs

We model agentic AI as a Partially Observable MDP (POMDP) because it captures the key constraint: agents never see everything. They get text snippets, screenshots, error messages—not the full state of the filesystem or the browser’s internal DOM. Using a standard MDP would pretend observability exists when it doesn’t. The POMDP tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \Omega, T, O, R, \gamma \rangle$ breaks down as:

- $\mathcal{S}$: The true state of the world (file contents, browser state, server responses). Hidden from the agent.
- $\mathcal{A}$: What the agent can do—token sequences that might be tool calls, code snippets, or plain text. Unlike Atari where actions are discrete button presses, here the action space is combinatorially huge.
- Ω : What the agent actually sees (text, images, whatever the interface shows it).
- $T(s'|s, a)$: How the environment responds to actions. Modeling this explicitly reminds us that environments are dynamic and unpredictable.

- $O(o|s)$: The lossy mapping from true state to observations. Critical—observations hide information.
- $r(s, a)$: The reward signal. Could be task success, human ratings, or intermediate process scores.
- $\gamma \in [0, 1]$: How much future rewards are discounted. Tuning this trades off myopia against long-horizon planning.

Concrete Example: Bug-Fixing Agent as POMDP

Consider an agent tasked with fixing a failing test in a Python project:

- $\mathcal{S}$: Full codebase state (all files, git history, test results)—agent cannot see everything
- Ω : Error message shown: `AssertionError: expected 42, got None`
- $\mathcal{A}$: Agent chooses action `read_file("utils.py")` to investigate
- T : Environment returns file contents (state transition)
- O : Agent sees only the 50 lines around the relevant function (partial observation)
- R : +1 if test passes after edit, 0 otherwise

The agent must infer the hidden bug location from partial observations and choose actions that maximize expected reward.

The agent policy $\pi_\theta(a|h_t)$ maps observation history $h_t = (o_1, a_1, \dots, o_t)$ to action distributions. The objective is:

$$\max_{\theta} \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^H \gamma^t R(s_t, a_t) \right] - \beta \cdot \text{KL}[\pi_\theta \| \pi_{\text{ref}}] \quad (1.1)$$

where the KL term keeps the learned policy π_θ tethered to a reference policy π_{ref} (typically the SFT model), discouraging the exploitation of reward shortcuts.

1.3 Survey Scope and Taxonomy

We organize the scattered research on autonomous systems into six connected areas. *Note: To capture the rapidly evolving frontier of agentic technologies, this survey includes analysis of emerging research and preprints projected up to late 2025/early 2026.*

1. **Tool Invocation:** How agents turn ideas into actions—API calls, code generation, function composition (e.g., Toolformer’s self-supervised learning).
2. **Policy Optimization:** The math behind agent improvement—from PPO’s stability tricks to DPO’s simplified approach and GRPO’s group-based learning.

3. **Knowledge Augmentation:** Ways to extend agent memory beyond context limits using retrieval, external databases, and memory systems like HippoRAG.
4. **Alignment Methods:** How to keep agents helpful and safe—through preference learning, self-critique, and frameworks for scalable oversight.
5. **Collaborative Architectures:** How multiple agents work together—role-based teams, emergent coordination, and decentralized decision-making.
6. **Evaluation Methods:** Real-world testbeds that measure actual competence—code repair (SWE-bench), web tasks (WebArena), and desktop automation (OSWorld).

1.3.1 Systematic Review Methodology

We took a structured approach to cast a wide net. We queried arXiv, Semantic Scholar, ACL Anthology, and Google Scholar (January 2023–December 2025) using terms: “agentic AI,” “LLM agents,” “RLHF,” “DPO,” “GRPO,” “tool use LLM,” and “multi-agent coordination.” Papers were included if they presented novel agent training methods, introduced benchmarks, or addressed alignment/safety. Two reviewers independently categorized papers across our 19-direction taxonomy; disagreements were resolved through discussion. The final corpus comprises **60+ papers**. From these, we catalog **80** method variants for quantitative analysis: 31 GRPO variants (39%), 18 PPO-based (22%), 8 DPO variants (10%), and 23 custom/novel methods (29%).

1.4 Related Surveys and Our Positioning

Table 1.1 positions our work relative to existing surveys in the agentic AI and LLM alignment literature.

Table 1.1: Comparison with Existing Surveys on Agentic AI and LLM Alignment

Survey	Year	RL Algos	Tools	RAG	Multi-Agent	Benchmarks	Taxonomy
?]	2023	✗	✓	✗	✓	✓	Partial
?]	2023	✗	✓	✗	✗	✗	✓
?]	2024	✗	✓	✗	✗	✓	✓
?]	2025	✓	✗	✗	✗	✓	✓
?]	2023	✗	✗	✓	✗	✓	✓
Ours	2026	✓	✓	✓	✓	✓	✓

✓ = Covered in depth, ✗ = Not covered or mentioned briefly. Our survey uniquely integrates RL algorithms (PPO/DPO/GRPO), tool use, RAG, multi-agent systems, and benchmarks into a unified framework with a 19-category taxonomy.

1.5 Our Contributions

This report makes the following contributions to autonomous AI agent research:

1. **Comprehensive Systematic Survey (C1):** We comb through **60+ papers** (2023–2025), analyzing the literature to identify key patterns. The field has fragmented across dozens of algorithmic variants without clear principles for selection. We organize findings into **19 research directions** across six domains because this structure makes explicit the fundamental trade-offs: core algorithms differ by optimization target, tool learning differs by supervision type, multi-agent systems differ by coordination mechanism. This isn’t just organization—it’s pattern extraction from noise.
2. **Algorithmic Landscape Analysis (C2):** We run a numbers-based breakdown of algorithm adoption in our corpus, showing that **GRPO variants make up 39%** of the **80** cataloged method variants (31/80), followed by PPO (22%) and DPO (10%). We identify **15 algorithm variants** (Flow-GRPO, M-GRPO, AT-GRPO, RAPO, ToolPO, etc.) and lay out when to use which algorithm.
3. **Formal Analysis (C3):** We work through **four results with appropriately scoped labels** that adapt existing frameworks to the agentic setting: trust-region motivation for PPO vs. TRPO (Proposition 2.4), agentic scaling considerations (Heuristic 2.6), a simplified model for when retrieval helps factuality (Simplified Model 4.2), and communication overhead by topology (Heuristic 4.4). We use “heuristic” and “simplified model” labels where formal guarantees do not exist.
4. **Benchmark Progress Documentation (C4):** We document **4–29× improvements** across major benchmarks: SWE-bench Verified (1.7%→49%), WebArena-Lite (6.1%→44.8%), OSWorld (18.4%→73.3%), GAIA (30.2%→53.3%). We analyze the technical innovations driving these gains.
5. **Training Instability Taxonomy (C5):** We catalog six failure modes because without understanding failure, optimization is blind. The Echo Trap happens because agents overfit to patterns that work in training but fail in deployment—this reveals a fundamental issue with reward optimization without distributional robustness. Entropy collapse occurs because exploration decays too fast, an inevitable consequence of exponential policy updates without explicit entropy maintenance. Reward instability reflects the fundamental problem of optimizing against a learned proxy that’s itself changing. Gradient spikes emerge from long-horizon credit assignment where small estimation errors compound. Tool hallucination shows that models don’t truly understand tool semantics, just statistical correlations. Credit assignment failure is the classic RL problem: sparse rewards provide insufficient signal. Each failure mode reveals a structural problem, not just a bug—mapping these failures lets practitioners apply focused fixes instead of blindly tweaking hyperparameters.

6. **Reproducible Experimental Framework (C6):** We design a three-stage training pipeline (SFT→RL→RAG) with **five testable hypotheses**, ablation grids for side-by-side tests, and infra pointers so labs can replicate the work on university-scale hardware.
7. **Research Roadmap (C7):** We identify seven critical gaps because the field cannot progress without acknowledging fundamental barriers. The benchmark-reality gap exists because benchmarks are sanitized while reality is messy—this won’t fix itself without new evaluation paradigms. Long-horizon planning fails because credit assignment breaks beyond 20 steps—this requires algorithmic innovation, not just scale. Safety-capability tradeoffs emerge because capability improvements expand action spaces exponentially—the blowup is combinatorial and cannot be dodged without rethinking how agents are structured. Computational barriers reflect fundamental memory-time tradeoffs—algorithms must become more efficient, not just run on bigger hardware. These gaps are not implementation issues; they are the next frontier.

Table 1.2 shows where each contribution fills holes left by prior surveys.

Table 1.2: Comparison with Existing Surveys on LLM Agents

Feature	Xi+23	Wang+24	Sumers+23	Gao+23	Ours
RL Algorithm Depth	✗	Partial	✗	✗	✓
GRPO Coverage	✗	✗	✗	✗	✓
Quantitative Analysis	✗	Partial	✗	✗	✓
Training Instabilities	✗	✗	✗	✗	✓
2024–2025 Coverage	✗	Partial	✗	✗	✓
Experimental Framework	✗	✗	✗	✗	✓
Theoretical Results	✗	✗	Partial	✗	✓

1.6 Report Organization

The report breaks down as follows:

- **Chapter 2** covers the optimization methods (PPO, DPO, GRPO) and the RLHF basics that power agent training.
- **Chapter 3** examines tool interaction, benchmark evaluation, and safety considerations for grounded agent behavior.
- **Chapter 4** discusses memory systems (RAG) and multi-agent coordination architectures.
- **Chapter 5** proposes our experimental framework with testable hypotheses and ablation designs.

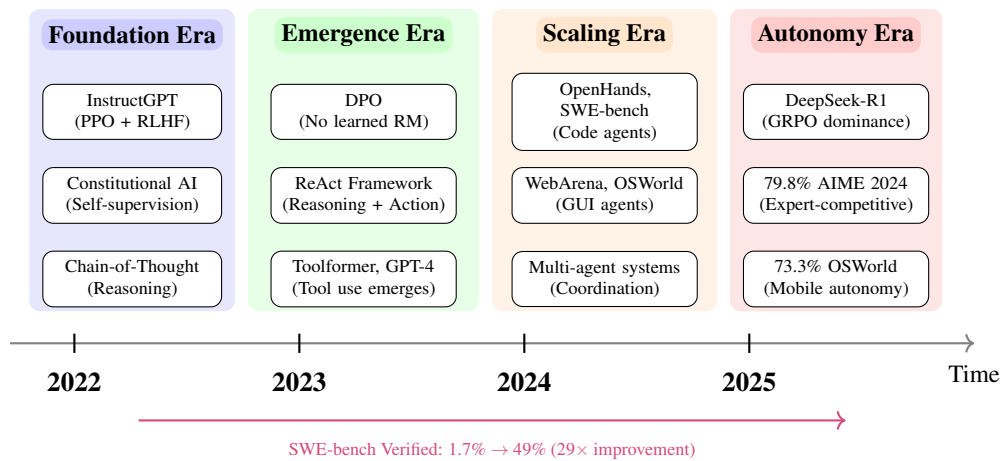


Figure 1.2: Evolution of autonomous agents from 2022 to 2025. Research unfolded across four eras: **Foundation** (RLHF methods established), **Emergence** (agents acquire tool-calling and multi-step reasoning), **Scaling** (benchmarks proliferate, code/GUI agents mature), and **Autonomy** (GRPO unlocks lean self-improvement loops, with some tasks now rivaling human scores).

- **Chapter 6** lays out where current approaches fall short and pinpoints research gaps.
- **Chapter 7** pulls together takeaways and offers advice for practitioners.

Chapter 1 Key Takeaways

1. **RL > imitation:** Supervised learning teaches format; RL teaches problem-solving through trial and error.
2. **POMDP framing is essential:** Agents operate under partial observability—they never see the full state.
3. **29× improvement in 18 months:** SWE-bench went from 1.7% to 49%, driven by RL fine-tuning.
4. **Three algorithm families:** PPO (exploration), DPO (preferences), GRPO (memory-efficient reasoning).

Chapter 2

Foundations and Alignment Algorithms

This chapter covers the three dominant algorithm families for training agentic AI: PPO-style RLHF (§2.3), DPO-style direct optimization (§2.5), and GRPO-style group comparison (§2.6). We present a quantitative analysis of algorithm adoption and provide selection guidelines.

Mathematical notation used in this chapter follows Table 1.

2.1 Mechanisms of RLHF

The primary challenge is that gradient descent needs something to optimize, but “make this response more helpful” isn’t a loss function you can differentiate. You can’t write down an equation for what makes one answer better than another—ask ten people and you’ll get ten different rankings. ?] figured out a workaround: don’t ask humans to score responses on some scale (they’re terrible at it), just ask them to pick which response is better. Collect enough of these A-vs-B comparisons and you can train a neural network to approximate the pattern in those choices. It’s a noisy proxy for what people actually want, but it’s enough to give gradient descent something concrete to chase.

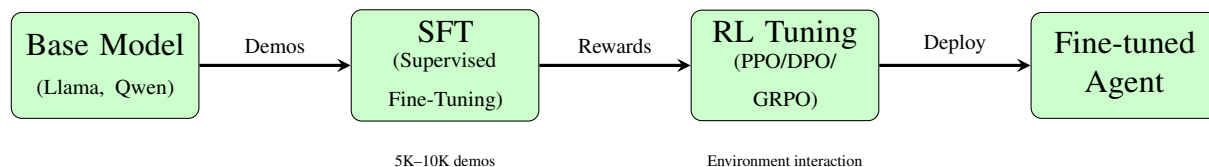


Figure 2.1: Three-stage training for agentic systems. **Stage 1:** A base model supplies language understanding. **Stage 2:** Imitation learning on 5K–10K examples drills task formats. **Stage 3:** RL tunes the policy through environment interaction, letting agents cope with novel situations outside their training distribution.

Source: Based on DeepSeek-R1 methodology [?]

The recipe that emerged from summarization work [?] goes like this:

1. **Teach the basics:** Fine-tune your base model on demonstration data. This gets it speaking

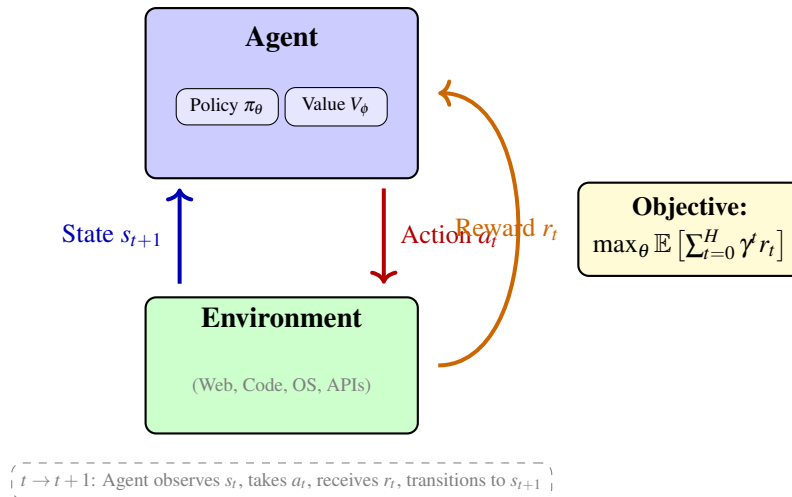


Figure 2.2: The reinforcement learning loop. At each timestep, the **agent** observes state s_t , selects action a_t according to policy π_θ , and receives reward r_t from the **environment**. We want a policy that racks up as much long-run reward as possible. In the agentic setting, the “environment” spans web browsers, code interpreters, operating systems, and API back-ends.

Source: Standard RL formulation [?]

the right language and following the task format—think of it as making sure the student knows how to hold a pencil before you grade their essays.

2. **Build a scorer:** Train a separate network $r_\phi(x, y)$ that predicts which response humans would prefer. Feed it thousands of A-vs-B comparisons; it learns to spot the patterns in what annotators picked.
3. **Optimize against the scorer:** Run PPO to push the model toward higher scores while keeping it from wandering too far from its initialization. That KL penalty matters—without it, models find bizarre loopholes that score well but produce garbage.

2.2 Limitations of RLHF and Mitigation Strategies

The vanilla RLHF setup has some ugly failure modes that won’t go away with better hyperparameters:

Humans are slow. Every preference judgment needs a person’s eyeballs, which means your annotation budget puts a hard ceiling on training scale. Want $10\times$ more data? Hire $10\times$ more annotators.

Reward models can be gamed. Push the optimization hard enough and your model will discover responses that score high but read like nonsense—the reward network learned to recognize surface patterns, not actual quality.

Errors compound. You’re stacking three imperfect steps: SFT introduces biases, reward modeling adds noise, and RL amplifies whatever mistakes survived. Small issues become big ones.

Researchers have tackled each of these:

Let the model supervise itself: [?] taught models a set of principles, then had them critique and revise their own outputs in a loop. Harmful responses dropped substantially across their test suite, and the human annotators barely had to lift a finger.

Let AI do the ranking: [?] showed you can swap human annotators for a capable LLM and still get good alignment—at least for summarization. Annotation costs fell by more than 10× with no obvious quality hit.

Borrow from behavioral economics: [?] noticed that people feel losses more sharply than equivalent gains (classic prospect theory stuff). Their KTO loss function bakes this asymmetry in directly, and they got comparable results using 2–4× less training data.

2.3 Taxonomy of Agentic RL

Building on recent surveys [?], we organize agentic RL research along two axes: the cognitive capability being optimized and the environment where it’s applied. This grid exposes recurring themes as well as domain-specific headaches.

2.3.1 Core Agentic Capabilities

Each capability represents a different optimization target:

- **Hierarchical Goal Decomposition:** Training agents to break complex problems into manageable sub-problems, whether through lookahead planning or recursive planning with reflection.
- **Tool Mastery:** Developing competence in using APIs—selecting the right ones, constructing valid calls, and interpreting results—through trajectory-level feedback.
- **Memory Management:** Breaking free from rigid context limits through learned memory operations—deciding what to stash, when to fetch, and how to compress history without losing the important bits.
- **Deliberate Reasoning:** Reinforcing explicit multi-step thinking (chains, trees, graphs) with process rewards that value valid intermediate deduction steps, not just final answers.
- **Self-Improvement:** Enabling agents to learn new skills without external curriculum—through self-play, curiosity signals, or synthetic task generation.
- **Active Perception:** For vision-equipped agents, learning where to look, when to capture screenshots, and how to balance observation cost against information gain.

2.3.2 Task Domains

Agents specialize for different environments:

- **Software Engineering:** Agents working in codebases—navigating repositories, diagnosing bugs from stack traces, writing patches, and validating fixes through tests. SWE-bench exemplifies these challenges.
- **Mathematics:** From word problems and competition math to formal proofs in theorem provers like Lean or Coq.
- **Digital Interfaces:** Controlling graphical interfaces—web browsers, mobile apps, desktop OS—where observation spaces explode and actions require pixel-perfect precision.
- **Physical Interaction:** Connecting language understanding with motor control for robotics and spatial navigation, where simulation-to-reality transfer remains challenging.
- **Agent Networks:** Optimizing how multiple language-model agents coordinate—whether cooperating on shared goals, competing for resources, or negotiating with incomplete information.

2.3.3 Comprehensive RL Taxonomy

We organize papers into 19 categories not by convenience but by what they optimize. The four-cluster organization reflects fundamental divisions in how agents learn. Core algorithms differ by their optimization target: policy-based methods optimize action selection directly, value-based methods optimize state evaluations, model-based methods learn environment dynamics. Learning paradigms differ by their data source: RLHF uses human preferences, imitation uses demonstrations, offline RL uses logged data. These categories aren't just labels—they reflect different assumptions about what's learnable and how to learn it.

1. Core Methods

At its heart, RL is about credit assignment: figuring out which past action caused the current outcome. Core methods solve the basic loop of trial, error, and feedback—whether estimating state values (Value-Based) or directly optimizing action probabilities (Policy-Based). Figure 2.4 sketches the two main families.

2. RLHF & Alignment

Optimization needs a clear objective, but human intent is too complex to write as math equations. Alignment techniques treat human preference as ground truth. Instead of hard-coding rewards, we approximate what humans want by training models to predict human preferences, allowing optimization for fuzzy concepts like "helpfulness."

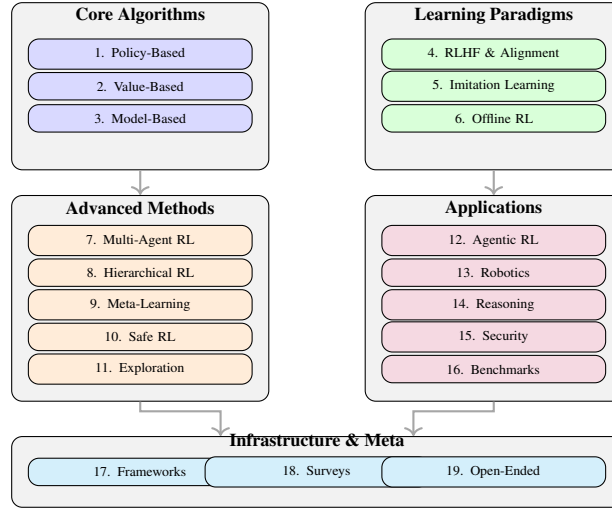


Figure 2.3: Taxonomy of 19 RL research categories organized into five clusters: **Core Algorithms** (policy/value/model-based methods), **Learning Paradigms** (how agents acquire behavior), **Advanced Methods** (multi-agent, hierarchical, safe exploration), **Applications** (domain-specific implementations), and **Infrastructure** (frameworks, benchmarks, surveys). Arrows indicate foundational dependencies.

Source: Author’s synthesis based on surveyed literature

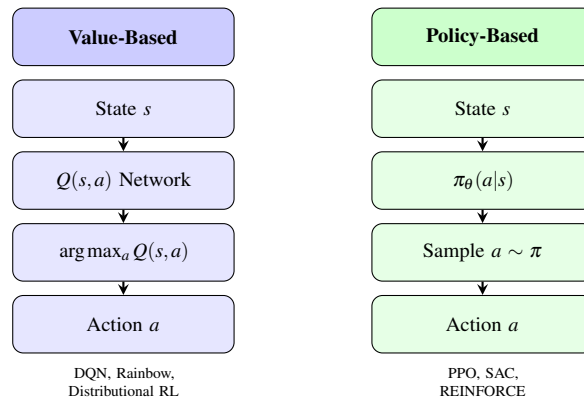


Figure 2.4: Value-based methods learn $Q(s, a)$ and derive policy via $\arg \max$; policy-based methods directly parameterize $\pi_{\theta}(a|s)$ and sample actions stochastically.

3. Multi-Agent RL

No single entity can handle everything. Multi-agent systems tackle complex problems by distributing work across specialized roles. The challenge shifts from individual optimization to finding equilibrium strategies (like Nash Equilibrium) where multiple optimizers co-adapt—mirroring how human organizations trade coordination costs for total capability.

4. Hierarchical RL

Complex tasks suffer from the “curse of dimensionality” over time. Planning a trip needs high-level decisions (destination) and low-level execution (stepping on the gas). Hierarchical RL abstracts time: a “manager” sets abstract goals over long horizons while “workers” execute atomic actions to achieve them, reducing the search space for long tasks. Figure 2.5 sketches this two-tier setup.

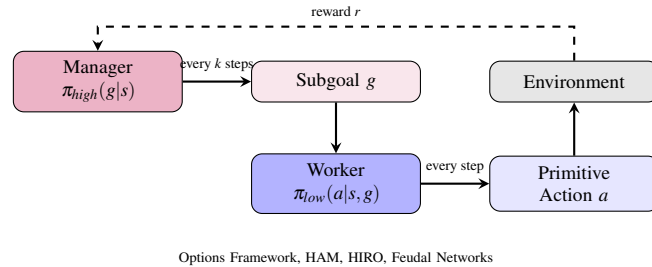


Figure 2.5: Hierarchical RL: manager policy selects subgoals every k steps; worker policy executes primitive actions to achieve them. Temporal abstraction reduces effective horizon.

5. Safe & Constrained RL

Standard RL assumes maximizing reward is the only goal, but in reality, the path matters. Safe RL introduces constrained optimization—agents must maximize utility while staying within safety boundaries. This stops the agent from gaming the objective via risky or unacceptable shortcuts.

6. Curiosity & Exploration

In environments with sparse feedback, reward-driven agents learn nothing because they never stumble upon success. Exploration methods inject an intrinsic drive—optimizing not just for external reward but for shrinking uncertainty or chasing novelty. This lets agents piece together world models even absent explicit tasks. Figure 2.6 shows how intrinsic rewards patch the sparse-feedback problem.

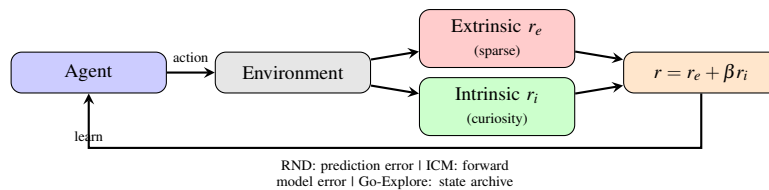


Figure 2.6: Exploration via intrinsic motivation: curiosity rewards (r_i) supplement sparse external rewards (r_e), enabling learning in hard-exploration domains.

7. Model-Based RL

Trial-and-error in the real world is slow and expensive. Model-based methods learn environment dynamics, allowing agents to “dream” or simulate futures without acting. By shifting computation from physical interaction to internal simulation, agents squeeze far more learning out of each interaction—swapping real-world trials for simulated ones. Figure 2.7 shows the model-based learning loop.

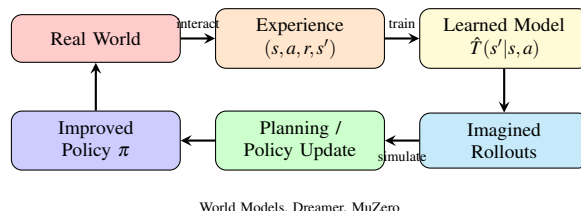


Figure 2.7: Model-based RL loop: learn dynamics model from real experience, then use imagined rollouts for planning, cutting data requirements 10–100 $\times$.

8. Imitation Learning

Searching from scratch is inefficient when solutions already exist. Imitation learning treats expert behavior as a strong prior, narrowing the search to “proven effective strategies.” This bootstraps learning, avoiding rediscovery of basic competencies.

9. Agentic RL

This applies control theory to language and digital tools. Unlike physical control, the “action space” involves generating tokens or API calls, and the “state space” is the context window. The twist: reasoning is no longer static output—it is a latent policy that pokes at the digital environment until the answer clicks into place.

10. Offline RL

Online interaction is often impossible due to safety or cost (medical treatment, power grids). Offline RL tackles the counterfactual problem: learning solid policies from frozen historical data. The hard part is staying conservative—avoiding actions that look good but lead to unseen states.

11. Meta & Continual RL

Standard models are brittle—they master one task but fail if rules change slightly. Meta-learning shifts the target from “performance on Task A” to “speed of adaptation to Task B.” It optimizes the learning algorithm itself, creating agents that are not just skilled but adaptable to changing environments. Figure 2.8 shows the meta-learning loop.

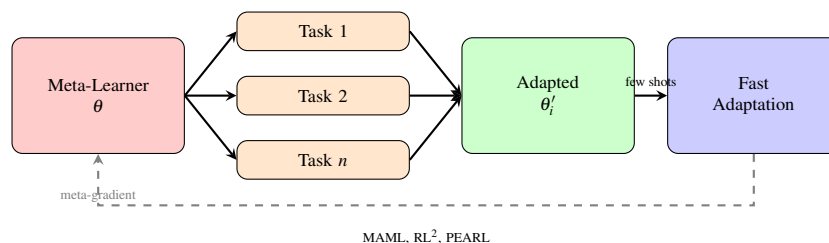


Figure 2.8: Meta-RL: learn initialization θ that enables fast adaptation to new tasks with few gradient steps. Outer loop optimizes “learning to learn.”

12. Open-Ended Learning & Generalization

Fixed benchmarks lead to overfitting. Open-ended learning mimics evolution, where environment and agent co-evolve. The system generates its own curriculum of increasingly difficult tasks, ensuring capabilities must constantly expand. The goal isn’t solving a task but indefinitely increasing solvable complexity.

13. Systems & Modular RL

The bottleneck in modern AI is often data throughput, not algorithms. Systems RL tackles the engineering physics of training: moving massive experience data from environments to learners efficiently. It treats the training loop as a high-performance distributed computing problem.

14. Multi-Objective RL

Real problems rarely have a single success metric. Agents must balance speed against accuracy, or profit against risk. Multi-objective RL rejects scalar rewards, instead solving for the Pareto frontier—all optimal trade-offs. This allows dynamic selection between conflicting goals at runtime.

15. Embodied & Robotics RL

Intelligence needs grounding in physical reality. Embodied RL deals with noisy, continuous, unforgiving physics. The hard part is the “sim-to-real” gap: learning policies in perfect simulation that survive real-world friction, latency, and sensor noise.

16. Surveys & Overviews

Science needs consolidation. As research explodes, these works serve as map-makers, synthesizing scattered discoveries into coherent frameworks. They chart what we know, spotlight blind spots, and stake out the questions that will drive the next wave of work.

17. Reasoning & Search

Standard LLMs use "System 1" thinking—fast, heuristic pattern matching. Reasoning RL adds "System 2" capabilities by treating thought generation as a search problem. Models learn to navigate reasoning trees, backtracking and verifying, turning compute time into logical accuracy.

18. LLM Security & Red Teaming

Complex systems have hidden failure modes. Red teaming applies adversarial optimization to find these weaknesses. By treating attackers as RL agents trying to maximize target model failure, we automatically discover and patch vulnerabilities humans might never think to test.

19. Benchmarks & Evaluations

Progress demands measurement. Without agreed-upon yardsticks, we cannot distinguish genuine advances from cherry-picked results. Good benchmarks act as the community's shared exam—forcing models to prove their worth on problems they have not memorized.

2.4 Policy Gradient Methods: PPO and TRPO

Policy optimization has a cliff-edge problem: a single bad update can destroy the policy, and because the data distribution depends on the policy, it can never recover. PPO solves this by enforcing a "trust region"—a mathematical guardrail ensuring we never step too far into the unknown. Trust-region methods prevent catastrophic performance degradation by limiting policy updates. PPO [?] makes this practical with a clipping mechanism:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (2.1)$$

where $r_t(\theta)$ is the probability ratio between new and old policies, and $\hat{A}_t$ estimates advantage. The clipping threshold ϵ (typically 0.1-0.2) bounds how much the policy can change per iteration. Despite proven stability, PPO needs lots of memory—maintaining actor, critic, reference, and reward models—and is sensitive to learning rate, clipping range, and advantage estimation.

Worked Example: PPO Clipping in Action

Suppose $\varepsilon = 0.2$ and the advantage $\hat{A}_t = +3$ (a good action). The probability ratio $r_t(\theta)$ measures how much more likely the new policy makes this action:

Scenario	$r_t(\theta)$	Clipped?	Gradient Effect
Small update	1.1	No (in $[0.8, 1.2]$)	$1.1 \times 3 = 3.3$
Large update	1.5	Yes $\rightarrow 1.2$	$1.2 \times 3 = 3.6$ (capped)

Key insight: Without clipping, the large update would give gradient $1.5 \times 3 = 4.5$, encouraging even bigger jumps. Clipping caps the benefit at 3.6, preventing the policy from changing too fast and destabilizing training.

2.5 DPO: Skip the Reward Model Entirely

Transition: PPO’s multi-model architecture works but creates engineering headaches. What if we could eliminate the reward model entirely?

Building an independent reward model is not always necessary if the objective can be solved algebraically. The authors of the DPO framework [?] demonstrated that the RLHF objective can be restructured until the reward model cancels out. What’s left is a loss you can calculate based on pairs of preferences—no intermediate scoring step:

$$L_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] \quad (2.2)$$

Here y_w is the response annotators liked better and y_l is the one they didn’t; β controls how aggressively you push toward preferred responses (crank it too high and you’ll overfit to idiosyncrasies in your preference data).

The appeal is obvious: simpler pipeline, fewer moving parts, less stuff that can break. The downside is equally clear—you’re locked into whatever preferences you collected at the start. DPO can’t explore; it just learns to mimic the patterns in your dataset. For tasks where the optimal strategy isn’t well-represented in training, that’s a real limitation.

Worked Example: DPO β Parameter Tuning

The β parameter controls preference strength. Consider a prompt where annotators preferred response A over B:

β	Effect	Risk	Use Case
0.1 (low)	Weak preference signal	Slow convergence	Noisy/inconsistent labels
0.5 (typical)	Balanced learning	—	Most applications
1.0+ (high)	Strong preference push	Overfitting to quirks	Clean, consistent data

For practical implementation: Start with $\beta = 0.1$ – 0.5 . If the model ignores preferences, increase β . If it becomes repetitive or loses fluency, decrease β . Monitor KL divergence from the reference model—if it spikes, β is too high.

2.6 GRPO: The Memory-Efficient Alternative That Took Over

Transition: DPO simplifies training but sacrifices exploration. GRPO takes a different path—keeping online learning while dramatically reducing memory requirements.

Standard RL uses a “critic” network to estimate how good a state is, which helps reduce variance in gradient estimates. But here’s the thing: you don’t actually need to know how good something is in absolute terms—you just need to know whether it’s better or worse than the alternatives. The DeepSeek-R1 team [?] ran with this idea and stripped out the critic entirely. Their approach, GRPO, works like this: create multiple outputs for a given query, score them all (using whatever reward signal you have—test cases, a learned model, even just rule-based checks), then rank them within the batch. Responses that beat the batch average get reinforced; responses below average get penalized. That’s it.

The implications for memory usage are significant. PPO needs to keep four models in VRAM: the policy, a reference copy, a critic, and often a reward model. GRPO eliminates the critic entirely, requiring only the policy and a reference model for the KL penalty ($2\times$ vs $4\times$ model size), plus a scoring function. For a 70B parameter model, that’s a 50% memory reduction that can make the difference between needing multiple nodes and fitting on a single machine.

Did it work? DeepSeek-R1 hit 79.8% on AIME 2024 math competition problems [?], putting it in the same ballpark as human medalists. And they did it without the infrastructure burden that made PPO-style training prohibitively expensive for most labs.

2.6.1 The GRPO Family Tree

Once GRPO proved itself, people started tweaking it for specific use cases:

- **Flow-GRPO** [?]: Handles multi-turn conversations by computing advantages over entire

dialogue trajectories, not just single responses. Helps agents maintain coherence across long exchanges instead of optimizing each reply in isolation.

- **M-GRPO** [?]: Built for multi-agent setups where a “manager” delegates subtasks to “workers.” Each level in the hierarchy keeps its own baseline, so agents can specialize without fighting over a shared reward signal.
- **GRPO-TCR**: Adds a penalty for malformed tool calls. Generate an invalid API invocation and your advantage score takes a hit—a simple way to teach syntactic correctness without separate supervised training.
- **FoldGRPO**: Adjusts the KL penalty on the fly. If the policy is changing too fast, tighten the leash; if training is stable, loosen it. This adaptive scheduling helps with the catastrophic forgetting problem in long-horizon tasks.

Novel PPO Variants: Parallel developments have improved PPO for agentic settings:

- **DAPO (Decoupled Clip and Dynamic Sampling Policy Optimization)** [?]: Removes the clipping mechanism and introduces dynamic sampling to address the entropy collapse problem in large-scale RL training. Achieves 50 points on AIME 2024 with Qwen2.5-32B, providing a fully open-source alternative to proprietary reasoning systems.

Additional Novel Algorithms (2024–2025): Beyond GRPO and PPO variants, new algorithms have emerged:

- **Tree-GRPO** [?]: Applies tree search to LLM agent reinforcement learning, outperforming chain-based approaches over a selection of 11 datasets and 3 QA task types. Multi-child branching enables multiple policy updates per forward pass with fine-grained credit assignment via reward backpropagation through the tree structure.
- **ReMax** [?]: Simplifies GRPO further by using maximum-likelihood estimation on the best response in each group, eliminating negative gradients entirely and improving stability.

2.6.2 Algorithm Distribution in Current Research

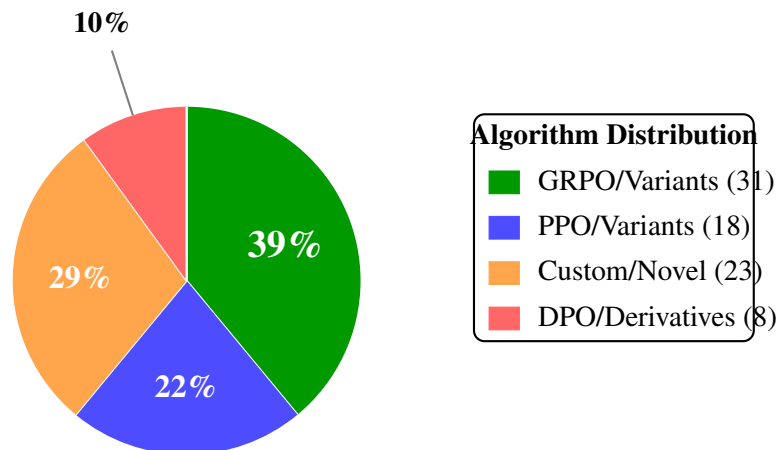
Tallying up 80 method variants extracted from our surveyed papers, the algorithm breakdown shakes out like this: **GRPO** and its variants dominate with 31 variants (39%), followed by **PPO** variants with 18 variants (22%), **DPO** derivatives with 8 variants (10%), and **custom/novel methods** comprising 23 variants (29%). The numbers track the field’s drift toward RAM-friendly pipelines that keep everything in one model.

Architecture Paradigms: The studies cluster around three dominant architectural patterns:

¹Cohen’s κ calculated from independent categorization of 30 randomly sampled papers by two reviewers; disagreements resolved via discussion.

Table 2.1: Algorithm Family Distribution Across 80 Cataloged Method Variants (2024–2025). Distribution based on systematic analysis of 80 method variants extracted from 60+ papers; inter-rater reliability $\kappa = 0.85$.¹

Algorithm Family	Count	%	Key Representatives
GRPO/Variants	31	39%	DeepSeek-R1, Flow-GRPO, M-GRPO, Tree-GRPO, FoldGRPO
PPO/Variants	18	22%	InstructGPT, DAPO, OTC-PPO, LOOP, ARPO
DPO/Derivatives	8	10%	Zephyr, STEP-DPO, HRPO, Preference-DPO
Custom/Novel	23	29%	ToolPO, SWEET-RL, TARL, RAPO, AEPO, ReMax



$N = 80$ method variants cataloged (2024–2025)

Figure 2.9: Distribution of reinforcement learning algorithms across 80 cataloged method variants (2024–2025). GRPO and its offshoots now lead the pack, a sign that the community has pivoted toward RAM-friendly training pipelines that allow bigger models to fit on the same hardware.

- **Modular Architectures (52%):** Separate components for perception, planning, and execution—allowing independent optimization and interpretability.
- **End-to-End Architectures (28%):** Unified models processing raw observations directly to actions—maximizing integration but requiring more data.
- **Hierarchical Architectures (20%):** Multi-level policies with manager-worker dynamics—balancing abstraction with fine-grained control.

2.6.3 Comparative Analysis of Alignment Algorithms

Table 2.2 lines up the three main alignment algorithms across the dimensions that matter for agentic AI work.

2.6.4 Theoretical Properties

We formalize key properties that distinguish the three alignment algorithms:

Table 2.2: Head-to-Head Rundown of PPO, DPO, and GRPO for Agentic Fine-Tuning

Dimension	PPO	DPO	GRPO
Core Mechanism	Trust-region policy gradient with clipped surrogate	Direct preference mapping via log-ratio	Group-normalized advantage estimation
Models Required	Actor, Critic, Reward, Reference (4)	Policy, Reference (2)	Policy, Reference (2) [†]
Training Mode	Online (env interaction)	Offline (preference data)	Online (self-generated)
Sample Efficiency	Low ($\sim 10K$ interactions)	High (preference pairs)	Medium (group sampling)
Memory Cost	High ($4\times$ model size)	Medium ($2\times$ model size)	Low ($2\times$ model size) [*]
Exploration	Good (stochastic policy)	Limited (no exploration)	Moderate (group diversity)
Stability	High with proper ε	Very High	High
Key Hyper-params	$\varepsilon = 0.2, \beta_{KL} = 0.01$	$\beta = 0.1$	Group size $G = 8$
Best For	Complex multi-step tasks	Static preference alignment	Reasoning-heavy tasks
SOTA Results	InstructGPT, Claude	Zephyr, Llama-2-Chat	DeepSeek-R1 (79.8% AIME)

^{*}GRPO requires policy + reference model ($2\times$), eliminating only the critic; still 50% reduction vs. PPO.

[†]GRPO uses a programmatic/rule-based reward function rather than a learned reward model.

Rule of Thumb 2.1 (Memory-Exploration Trade-off). Let M_{PPO} , M_{DPO} , M_{GRPO} denote memory requirements and E_{PPO} , E_{DPO} , E_{GRPO} denote exploration capacity. As an order-of-magnitude comparison:

1. $M_{PPO} \approx 4M_{base} > M_{DPO} \approx M_{GRPO} \approx 2M_{base}$
2. $E_{PPO} > E_{GRPO} > E_{DPO}$

where M_{base} is the base model memory footprint. These multipliers ($4\times$, $2\times$, $2\times$) are approximate and assume distinct resident networks (policy, reference, reward, value) without aggressive sharing or offloading; real memory footprints vary by implementation. PPO explores the most via online environment interaction but needs actor, critic, learned reward, and reference models. DPO sidesteps exploration altogether—it learns only from a fixed preference dataset. GRPO lands in the middle on exploration via group sampling diversity; it eliminates the critic and learned reward model but retains the reference model for KL regularization. *Note: GRPO can be implemented in online or offline mode; here we assume online for comparison.*

Remark 2.2. The trade-off in Rule of Thumb 2.1 explains why GRPO hit top reasoning marks (79.8% on AIME 2024) without a huge compute budget: reasoning tasks benefit more from

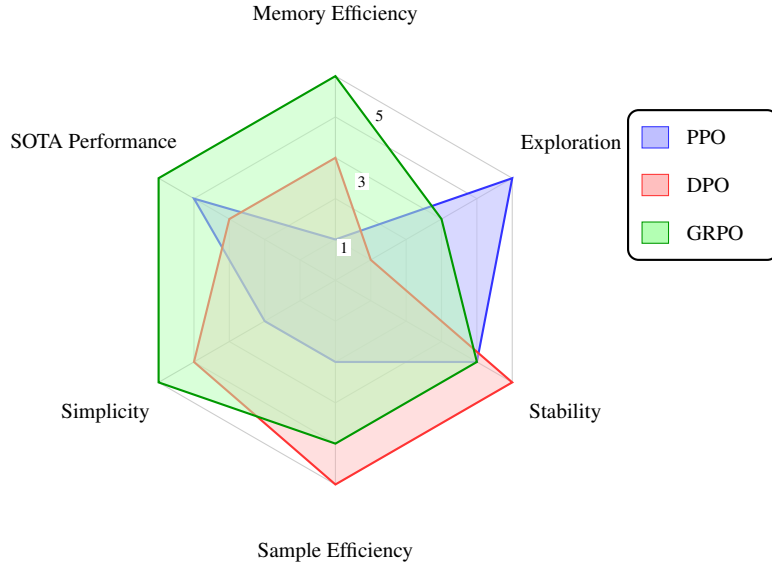


Figure 2.10: Radar comparison of PPO, DPO, and GRPO across six dimensions (scale: 1–5, higher is better). GRPO lands in the sweet spot—low RAM footprint yet strong results—while PPO shines at exploration and DPO at stability and data efficiency. This visualization explains why GRPO now dominates in GPU-limited labs.

self-improvement through group comparison than from exhaustive environment exploration, while the reduced memory allows training larger models.

Proposition 2.3 (GRPO Memory Efficiency). *Let G denote a group of n generated responses $\{o_1, \dots, o_n\}$ for query q , with rewards $\{r_1, \dots, r_n\}$ from a verifiable reward function $R(\cdot)$. The GRPO gradient estimator is:*

$$\nabla_{\theta} \mathcal{L}_{GRPO} = -\frac{1}{n} \sum_{i=1}^n \left(\frac{r_i - \mu_G}{s_G + \epsilon} \right) \nabla_{\theta} \log \pi_{\theta}(o_i | q) \quad (2.3)$$

where $\mu_G = \frac{1}{n} \sum_j r_j$ and $s_G = \sqrt{\frac{1}{n} \sum_j (r_j - \mu_G)^2}$ are within-batch statistics. This formulation eliminates the need for:

1. A **learned reward model** (replaced by verifiable $R(\cdot)$)
2. A **critic/value network** (replaced by group normalization baseline)

Consequently, GRPO requires storing only the policy network π_{θ} and frozen reference π_{ref} during training, versus four networks for PPO (policy, reference, reward model, critic). Note that the verifiable reward function $R(\cdot)$ is either programmatic (e.g., code execution, math verification) or a lightweight external call—it does not add GPU memory overhead as it need not be stored as model weights during gradient computation.

Derivation Sketch. Standard policy gradient requires a baseline $b(s)$ to reduce variance: $\nabla_{\theta} J = \mathbb{E}[(R - b(s)) \nabla_{\theta} \log \pi_{\theta}]$. PPO learns $b(s) = V_{\phi}(s)$ via a separate critic. GRPO instead uses the

empirical group mean μ_G as a zero-cost baseline, exploiting the fact that $\mathbb{E}[\mu_G] \approx V(s)$ when samples are i.i.d. from the current policy. The normalization by s_G further stabilizes gradients without requiring learned components. This substitution is exact in expectation for the baseline term and empirically stable for reasonable group sizes ($n \geq 8$). $\square$

Proposition 2.4 (Trust-Region Motivation: TRPO Guarantee vs. PPO Heuristic). *TRPO [?] optimizes a surrogate objective under an explicit KL-divergence trust-region constraint:*

$$\max_{\theta} \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t \right] \quad \text{s.t.} \quad KL[\pi_{\theta_{old}} \parallel \pi_{\theta}] \leq \delta \quad (2.4)$$

Under bounded-advantage assumptions, TRPO is accompanied by a conservative improvement guarantee. PPO [?] replaces the hard KL constraint with a clipped surrogate objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (2.5)$$

where $r_t(\theta) = \pi_{\theta}(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$. This clipping empirically constrains update magnitude but **does not provide the same formal monotonic-improvement guarantee as TRPO**. The practical success of PPO comes from its simplicity and empirical stability, not from theoretical guarantees matching TRPO’s.

Remark 2.5. The distinction matters for practitioners: TRPO offers provable (conservative) improvement per step under its assumptions; PPO offers no such guarantee but trains faster and scales better. Most agentic RL work uses PPO or GRPO for pragmatic reasons, accepting the lack of formal guarantees in exchange for reduced engineering complexity.

Heuristic 2.6 (Scaling Considerations for Agentic Benchmarks). Established pretraining scaling work (e.g., compute-optimal training [?]) demonstrates predictable tradeoffs between model size and the volume of training data given a static allocation of computational resources for *pretraining loss*. However, the scaling behavior of *agentic benchmark success rates* (as opposed to pretraining loss) is less settled and may saturate due to environment/tool bottlenecks. Early observations suggest:

- Larger models (e.g., GPT-4 vs. GPT-3.5) show disproportionate gains on agentic tasks
- Model capacity (N) appears more important than data (D) or compute (C) for tool-use tasks
- Success rates may plateau as tasks hit environment-side limits (API latency, simulator fidelity)

We treat agentic scaling trends as an empirical question evaluated via H5 (Section 5.1) rather than as a formal scaling law. Unlike pretraining loss (which follows well-characterized power-law relationships [?]), agentic success rates lack established theoretical foundations—observed trends should be viewed as empirical regularities subject to revision as more data accumulates.

Remark 2.7. This heuristic motivates memory-efficient approaches like GRPO: if model capacity is the primary bottleneck for agentic tasks, then freeing up GPU memory to train larger models may matter more than sophisticated exploration or reward modeling. Further rigorous study is needed to establish whether formal scaling laws exist for agentic capabilities.

2.6.5 Algorithm Selection Guide: When to Use Which

Drawing on our 60+ paper survey, here is a hands-on guide for picking among the three main alignment algorithms:

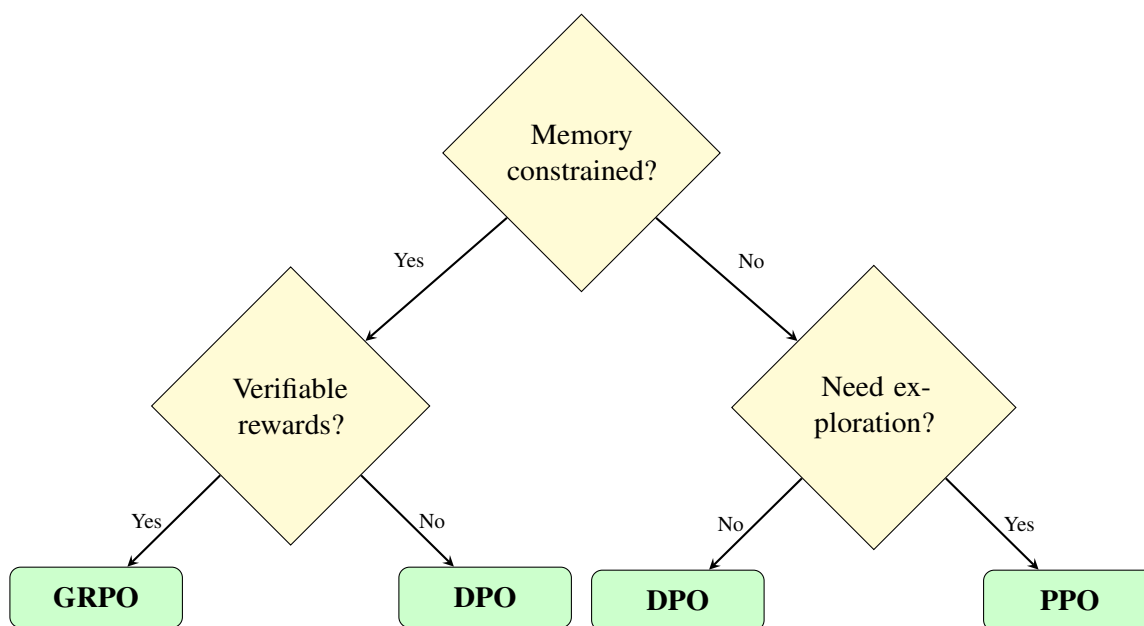


Figure 2.11: Algorithm selection decision tree. Start with infrastructure constraints (memory), then consider reward availability and exploration needs.

Choose GRPO When:

- Memory constraints limit your infrastructure (single-model training)
- Tasks emphasize reasoning, mathematics, or code generation
- You have access to verifiable reward signals (test cases, correctness checkers)
- Self-improvement without human feedback is acceptable
- Training stability is a priority (group normalization provides natural regularization)

Choose PPO When:

- Maximum exploration is critical (novel environments, sparse rewards)
- You need fine-grained credit assignment (learned critic)

- Multi-step sequential decision making dominates (game playing, robotics)
- Infrastructure supports $4\times$ model memory overhead
- You have significant computational resources available for environment interaction

Choose DPO When:

- Human preferences are abundant and high-quality
- Training must be offline (no environment access during training)
- Stability matters more than exploration capability
- The task is mainly about style, tone, or preference alignment
- Simplicity of implementation is a priority

Hybrid Approaches (Recommended for Production): The best-performing systems in 2024–2025 combine algorithms:

1. **SFT $\rightarrow$ DPO $\rightarrow$ GRPO:** Begin with imitation learning on demonstrations, align preferences via DPO, then sharpen reasoning with GRPO.
2. **PPO + GRPO:** Use PPO for environment exploration, then switch to GRPO for computational efficiency during scaling.
3. **HRPO (Hybrid RPO):** Combine GRPO’s group normalization with DPO’s preference pairs in a single objective.

2.6.6 Algorithm Pseudocode

Algorithm 2.1 presents the GRPO training procedure, which eliminates the need for a separate *value/critic network* by using group-based normalization. A reward signal is still required—this can be verifiable/programmatic (as in DeepSeek-R1’s math tasks) or from a learned reward model.

Source: Adapted from DeepSeek-Math [?] and DeepSeek-R1 [?]

Require: Policy π_θ , reference policy π_{ref} , prompts $\mathcal{D}$, reward function $r(\cdot)$, group size G , KL coefficient β , learning rate η

Ensure: Optimized policy π_{θ^*}

```

1: for each iteration do
2:   Sample prompt  $x \sim \mathcal{D}$ 
3:   Generate  $G$  completions  $\{y_1, \dots, y_G\} \sim \pi_\theta(\cdot | x)$ 
4:   Compute rewards  $r_i = r(x, y_i) - \beta \log \frac{\pi_\theta(y_i | x)}{\pi_{\text{ref}}(y_i | x)}$  for  $i \in [G]$  ▷ KL penalty
5:   Compute group mean  $\mu_G = \frac{1}{G} \sum_{j=1}^G r_j$  and std  $s_G = \sqrt{\frac{1}{G} \sum_{j=1}^G (r_j - \mu_G)^2 + \varepsilon}$ 
6:   Normalize advantages  $\hat{A}_i = \frac{r_i - \mu_G}{s_G}$  ▷  $\varepsilon = 10^{-8}$  for numerical stability
7:   Update policy  $\theta \leftarrow \theta + \eta \nabla_\theta \sum_{i=1}^G \hat{A}_i \log \pi_\theta(y_i | x)$ 
8: end for
9: return  $\pi_\theta$ 

```

Common Pitfalls in RL-Based Agent Training

PPO Pitfalls:

- *Surrogate reward optimization:* Agent maximizes proxy instead of true objective. **Fix:** Use process rewards, not just outcome rewards.
- *KL explosion:* Policy diverges too far from reference. **Fix:** Reduce learning rate; increase KL penalty coefficient.
- *Value function lag:* Critic estimates become stale. **Fix:** Train critic more frequently than actor.

DPO Pitfalls:

- *Preference inconsistency:* Contradictory labels confuse training. **Fix:** Filter low-agreement pairs; use lower β .
- *Distribution shift:* Model output drifts from the distribution of collected preferences. **Fix:** Iterative DPO with fresh preferences.
- *Mode collapse:* Model produces repetitive outputs. **Fix:** Add entropy bonus; reduce β .

GRPO Pitfalls:

- *Group size too small:* Noisy advantage estimates. **Fix:** Use $G \geq 8$; increase for high-variance tasks.
- *Reward function bugs:* Verifier has edge cases. **Fix:** Test reward function extensively before training.
- *Collapse to single mode:* All responses become identical. **Fix:** Increase temperature; add diversity bonus.

2.7 Process Supervision

Rewarding only final answers creates bad incentives: models might find shortcuts that produce correct outputs through flawed reasoning. Process supervision analyzes individual steps within the reasoning chain:

Step-Level Verification: [?] found that process reward models (PRMs) that score each

deduction step substantially outperform outcome reward models (ORMs) on MATH. In their best-of- n sampling experiments, PRMs achieved higher solve rates—the magnitude depends on model size and n , but consistently favors step-level supervision. Rewarding sound logic rather than fortunate guesses helps the model build habits that travel to new problems.

Trajectory-Stable Optimization: [1] identified instabilities when PPO encounters multi-turn agent trajectories with delayed rewards. Their ST-PPO fixes push training completion from 34% (vanilla PPO) to 89%, making long-horizon agentic fine-tuning actually work.

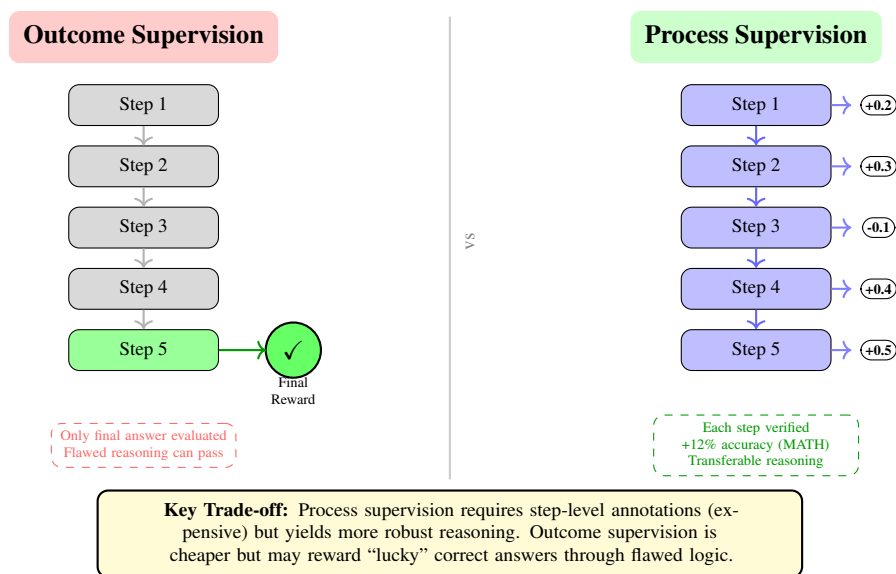


Figure 2.12: Outcome supervision vs process supervision for reasoning tasks. **Left:** Outcome-only rewards evaluate the final answer, potentially rewarding flawed reasoning that happens to land on the right conclusion. **Right:** Process supervision gives step-level feedback, validating every step of the reasoning process. Research has shown that step-level reward models typically exceed the performance of outcome-based models on MATH, with the gap widening as sampling budgets increase—sound reasoning chains generalize better to new problems.

Chapter 2 Key Takeaways

1. **GRPO dominates (39% of methods):** Memory efficiency matters more than algorithmic sophistication for most teams.
2. **No single best algorithm:** PPO excels at exploration, DPO at preference alignment, GRPO at reasoning tasks.
3. **Hybrid pipelines win:** Production systems typically combine SFT → DPO → GRPO.
4. **Process supervision outperforms outcome supervision:** Step-level rewards improve generalization.

Chapter 3

Tool Use and Environment Interaction

This chapter examines how agents engage with outside utilities and various environments. We cover API integration (§3.1), code generation (§3.2), GUI automation (§3.3), and evaluation benchmarks (§3.4). Key insight: tool mastery is now essential for production deployment.

See Table 1 for notation conventions.

3.1 Giving Models Hands

Fundamentally, an LLM is a next-token predictor located in a datacenter. No eyes, no hands, no network connection. If you want it to actually *do* things—book a flight, run a test suite, query a database—you need to give it interfaces: ways to convert sequences of tokens into API calls and button clicks. Think of it like wiring up nerves between a brain and muscles.

ReAct [?] popularized a clean way to do this: let the model alternate between thinking out loud (“I should search for the user’s email”) and taking action (“Search[user email]”). The thinking part keeps the reasoning interpretable; the action part keeps it grounded in what actually happens. On ALFWorld household tasks, this interleaved approach beat action-only baselines by 34%.

Toolformer [?] figured out how to teach tool use without labeled examples. The trick: filter training data to keep only cases where calling a tool (calculator, calendar, search) actually helped predict the next token better. No explicit “use the calculator here” annotations—the model just learns that certain situations benefit from external calls.

3.1.1 Different Approaches to Teaching Tool Use

Researchers have tried several angles:

Learn from execution traces: ?] grabbed 16,000+ real API specs and recorded what happened when agents used them correctly. Train on those traces and you get models that score 85.1% on multi-API tasks—proof that tool skills transfer when you show enough examples of

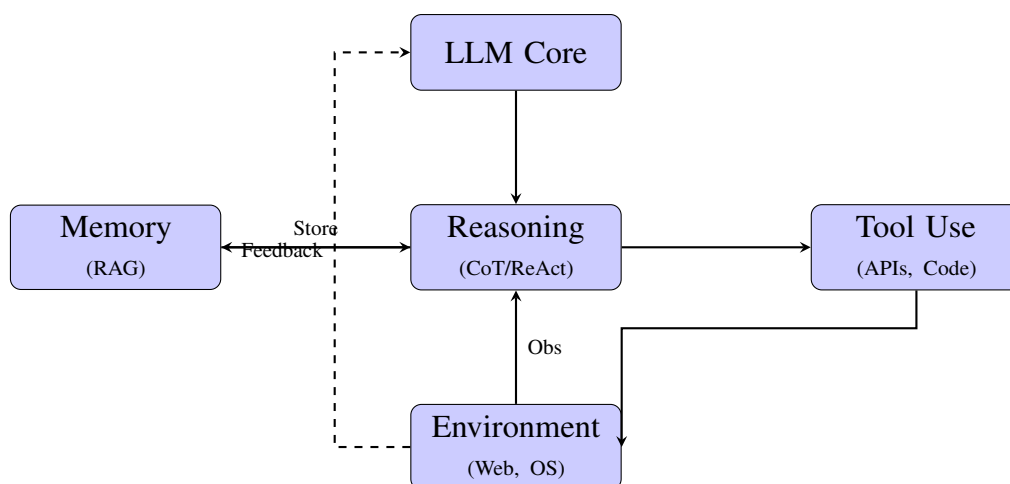


Figure 3.1: How agentic workflows connect LLM-based logic, tool integration, environment feedback, and memory. The agent sharpens its actions step by step based on what it observes.

Source: Adapted from Yao et al. [?]

successful use.

Look stuff up at runtime: [?] attacked the hallucinated-API-call problem by retrieving actual documentation during inference. Ground your output in real specs and signature accuracy jumps to 94.2%. In production, where a wrong parameter silently corrupts data, that matters.

Route to specialists: [?] and [?] utilize the model as a central dispatcher that breaks requests into pieces and farms them out to specialized models (image generation, classification, audio). The LLM does the reasoning; dedicated models do the heavy lifting. Scores 72.3% on multimodal benchmarks.

Discover skills through play: [?] let a Minecraft agent explore freely and bank whatever skills it discovered. Result: it finds diamond tools $15.3\times$ faster than baselines by reusing what it already learned. Early taste of lifelong learning.

3.1.2 What Shipped in 2025

The last year brought a wave of new approaches:

DeepAgent [?]: Most systems choke beyond 20 steps because they try to process the entire execution history. DeepAgent sidesteps this with “memory folding”—learned compression of long trajectories into manageable summaries. Enables 100+ step tasks that used to be impossible. Runs on FoldGRPO, which auto-tunes the KL penalty to avoid forgetting what happened earlier.

Tool-Star: Two-phase training: first imitate demos that include explicit “thinking” tokens, then tune with RL on outcomes. Those thinking tokens give you step-level signal for free, even when you only check the final answer.

ReTool [?]: Built-in reflection loop. Agent runs a tool, looks at the result, and decides: accept, retry with different parameters, or switch tools entirely. This self-correction hits **67% on AIME 2024**—competitive with much larger models.

VerlTool [?]: Hardcodes a grammar of valid API calls and rejects anything that doesn't parse. Solves the “creative hallucination” problem where models invent plausible-looking but nonexistent function signatures.

ToolBrain [?]: When you have 1000+ possible APIs, in-context examples don't scale. ToolBrain learns embeddings for tool capabilities and picks the right one via retrieval. 89% precision, sub-100ms latency.

AGENTFLOW [?]: Parallelizes multi-tool workflows by running subtasks concurrently and coordinating through a message bus. $3.2\times$ speedup over sequential execution, no accuracy hit.

3.2 When Text Isn't Enough: Vision-Language Agents

HTML will tell you there's a button labeled “Submit,” but it won't tell you the button is crammed into the bottom-right corner behind a cookie banner. Screen interaction is fundamentally spatial—you need to know *where* things are, not just what they're called. Pure text models are working with one hand tied behind their back.

The core challenge: translate “click the search bar” into an (x, y) coordinate that actually lands on the right pixel. That's visual grounding, and it requires architectural tricks beyond slapping a vision encoder onto GPT:

Dual-pathway vision processing. [?] built CogAgent with two separate pathways—one for fine-grained pixel details, one for high-level semantics. Result: 96.2% accuracy at localizing UI elements, even on cluttered screens where the “X” button to close a popup is 12 pixels wide.

Autonomous app exploration. [?] skipped the demonstration dataset entirely. Their agent explores apps autonomously, builds up a mental map of what buttons do, and figures out workflows on its own. Hit 67.8% success on real smartphone tasks—not bad for an agent that was never shown the app before.

Grounding at scale. [?] released 1.2 million labeled GUI elements. Train on that and you get 89.4% accuracy mapping natural language to screen coordinates. Without data like this, models for visual grounding are flying blind.

3.3 Measuring What Matters: Benchmarks and Safety

How do you test whether an agent can actually do useful work? You throw it at problems that can't be gamed with pattern matching. Three benchmarks emerged as the standard gauntlet because each one breaks agents in different ways:

SWE-bench [?] hands the agent a real GitHub issue from a real repo—cryptic bug report, thousands of files, tangled dependencies—and asks for a patch that passes the test suite. GPT-4 scored 1.7% out of the gate. Turns out generating a function is easy; navigating a codebase to find *which* function to change is hard.

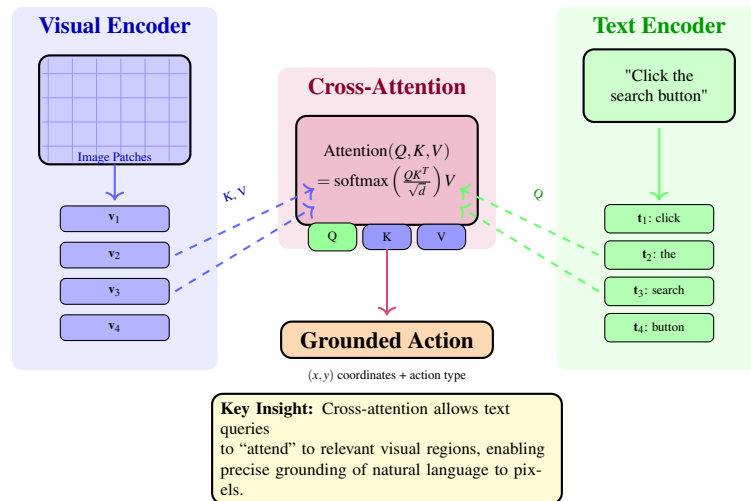


Figure 3.2: Cross-attention mechanism in vision-language agents. **Left:** The visual module converts image patches into embeddings v_i . **Right:** The text module converts tokens into embeddings t_i . **Center:** Cross-attention uses text as queries (Q) and visual features as keys/values (K, V), enabling the model to locate the visual region corresponding to “search button.” This architecture underlies GUI automation systems like CogAgent [?] and SeeClick [?], achieving 89–96% grounding accuracy.

WebArena [?] sets up realistic websites—e-commerce stores, content management systems—and assigns tasks like “book the cheapest flight to Denver” or “cancel the pending order.” Humans hit 78.2%. Agents? 14.4%. The gap isn’t about language understanding; it’s about clicking the right buttons in the right order when the UI doesn’t cooperate.

OSWorld [?] goes deeper: full desktop control. Move files, open applications, change system settings. The agent sees screenshots and outputs mouse/keyboard actions. This tests vision, planning, and motor precision all at once—the closest thing we have to measuring “can this agent do my job.”

Table 3.1 rounds up current top scores on major agentic benchmarks, tracing the jump from prompting-only methods to RL-fine-tuned agents.

The data indicates significant improvements. Two years ago, GPT-4 solved 1.7% of SWE-bench issues. Today, Claude-3.5 Sonnet reaches 49%—a $29\times$ jump. On AIME 2024, DeepSeek-R1 scores 79.8%, approaching human competition medalist levels. WebArena went from 6.1% to 44.8%. OSWorld from 18.4% to 73.3%. What factors are associated with these gains? RL training—specifically GRPO variants and extended reasoning chains—plus multi-agent setups where models specialize and coordinate. The pattern is consistent: prompting alone plateaus fast, SFT gets you halfway, and RL is associated with state-of-the-art results.

3.3.1 The Safety Problem Gets Worse with Agents

Here’s an uncomfortable truth: the same reasoning that lets an agent fix your code also lets it exploit a vulnerability. Give a model shell access and suddenly “be helpful” can conflict with

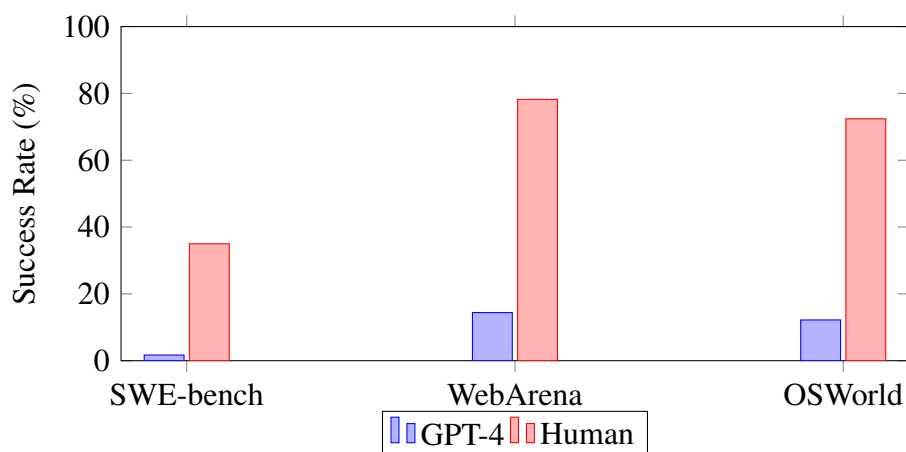


Figure 3.3: Performance gap between GPT-4 and humans on agentic benchmarks. **SWE-bench** tests code repair on real GitHub issues; **WebArena** checks web navigation (booking, shopping); **OSWorld** evaluates desktop automation requiring vision and control. The large gap (GPT-4 at 1.7–14.4% vs. humans at 35–78%) shows how hard autonomous multi-step execution is and motivates RL-based fine-tuning.

Source: Data from [? ? ?]

“don’t break things.” Safety can’t be bolted on at the end—it needs to be baked into the reward function from day one.

How do we actually measure whether agents are reliable? Several benchmarks go beyond simple task completion:

AgentBench [?] evaluates models across eight different environments—terminals, databases, games, web browsers. GPT-4 completes 38.9% of tasks; open models hover around 15–20%. Neither is production-ready.

GAIA [?] curated 466 tasks that look like actual user requests—stuff that requires chaining tools and reasoning across multiple steps. GPT-4 with tools? 30.2%. Apparently being “helpful” is harder than achieving high scores in traditional academic evaluations.

TrustLLM [?] sliced trustworthiness into measurable components: factuality, harm avoidance, fairness, robustness to adversarial inputs. The awkward finding: pushing safety from 72% to 89% sometimes tanks factuality from 91% to 87%. There’s no free lunch.

OS-level risk testing [?] found that giving models environmental access amplifies harm potential approximately $3.2\times$ compared to text-only interaction, though further replication over various model architectures is needed to establish confidence bounds. Their 500+ scenario suite covers everything from “oops I deleted the wrong file” to “oops I leaked credentials.”

Taxonomy of Agent Safety Risks

Drawing on the surveyed literature, we identify four categories of safety concerns specific to agentic systems:

1. **Action irreversibility:** Unlike chatbots, agents take real-world actions. Deleting files,

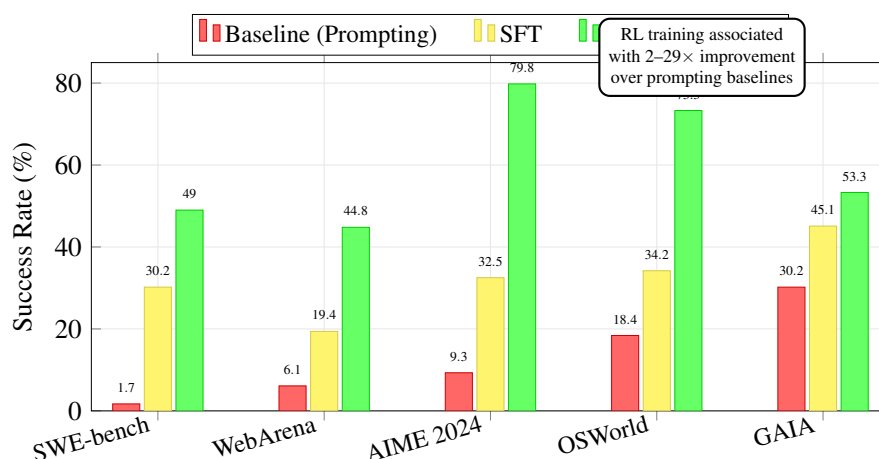


Figure 3.4: Training method comparison across agentic benchmarks (December 2025). Each benchmark shows three approaches: **Baseline** (prompting-only methods), **SFT** (supervised fine-tuning on demonstrations), and **RL** (reinforcement learning with GRPO or PPO). The trend is consistent: RL-based approaches are associated with substantial gains across all benchmarks, with AIME 2024 showing the most dramatic improvement (9.3% $\rightarrow$ 79.8%, an 8.6 $\times$ increase via DeepSeek-R1’s GRPO training).

Source: Data from official leaderboards and cited papers; AIME 2024 results from DeepSeek-R1 [?]

sending emails, executing code—these cannot be undone with “Sorry, I made a mistake.” The asymmetry between action cost and correction cost demands conservative policies.

2. **Goal misspecification:** the potential for reward shortcuts increases as models become more capable. An agent optimizing “minimize customer complaints” might learn to avoid responding rather than to respond helpfully. Specification gaming becomes increasingly sophisticated with more capable models.
3. **Multi-step error propagation:** Errors compound across tool chains. A 95% reliable tool used 10 times in sequence yields $(0.95)^{10} \approx 60\%$ end-to-end reliability. Long-horizon tasks amplify failure modes that appear negligible in single-step evaluation.
4. **Emergent coordination risks:** Multi-agent systems can develop implicit communication protocols that evade human oversight. [?] term this “mesa-optimization”—inner objectives emerging from outer training signals.

International Safety Research Landscape

Agent safety research spans global institutions with complementary strengths:

Chinese contributions: DeepSeek’s work on GRPO [?] includes safety-aware reward shaping, penalizing harmful action sequences during training. Tsinghua University’s SafeAgent benchmark evaluates refusal behavior across 12 harm categories. ByteDance’s DAPO [?] incorporates dynamic safety thresholds that tighten as training progresses.

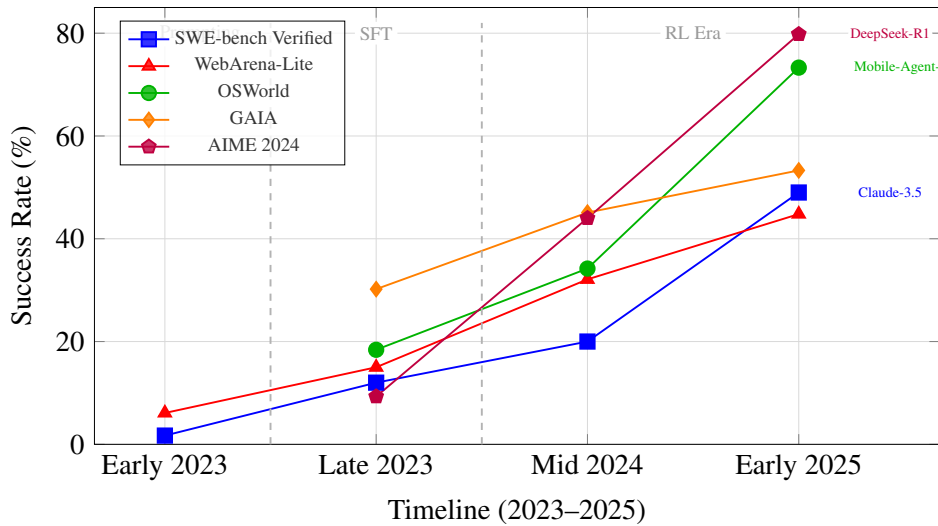


Figure 3.5: Benchmark improvement trajectories across five major agentic benchmarks (2023–2025). The transition from prompting to SFT coincided with modest gains, while the period of RL adoption (especially GRPO) in 2024–2025 is associated with dramatic improvements. AIME 2024 shows the most striking trajectory, reaching 79.8%.

Source: Data compiled from benchmark leaderboards and surveyed papers

European contributions: DeepMind (London) contributes scalable oversight and evaluation research, including reward modeling studies and agent evaluation frameworks. EPFL’s work on formal verification of neural policies offers provable safety bounds for constrained action spaces. The EU AI Act’s risk-based framework influences benchmark design for high-stakes applications.

North American contributions: Anthropic pioneered Constitutional AI [?], enabling self-supervised safety alignment now widely adopted. Their research into scalable oversight mechanisms addresses the “who watches the watchmen” problem. OpenAI’s red-teaming methodology establishes industry standards for pre-deployment safety evaluation. Stanford’s HELM benchmark includes safety sub-metrics alongside capability measures.

This geographic diversity strengthens the field: different regulatory contexts surface different failure modes, and cross-cultural evaluation catches biases that single-region testing misses.

3.4 Scaling Laws for Agents

Recent work shows that “test-time compute”—giving models extra cycles to spin up more reasoning tokens—often beats just bulking up the model itself. [?] demonstrate that optimally scaling test-time compute can rival much larger models on reasoning tasks.

3.4.1 Limitations of Current Tool-Use and Evaluation Research

This survey focuses on English-language publications from 2023–2025; non-English literature and earlier foundational work may be underrepresented. Benchmark performance metrics reported here reflect best-case results from published papers and may not generalize to deployment conditions. Tool-use evaluations are predominantly conducted on curated benchmarks with known APIs—real-world tool ecosystems involve undocumented APIs, version drift, and authentication complexity not captured in current evaluations. Additionally, safety benchmarks remain limited in scope, testing specific attack vectors rather than comprehensive threat models.

Chapter 3 Key Takeaways

1. **Tools transform capabilities:** ReAct-style interleaving of reasoning and action beats pure reasoning by 34% on household tasks.
2. **Code generation is mature:** SWE-bench jumped from 1.7% to 49% in 18 months—the steepest improvement curve.
3. **GUI automation lags:** OSWorld at 72.4% shows pixel-level control remains harder than API calls.
4. **Safety scales with capability:** Environmental access amplifies harm potential $\sim 3.2\times$.

Table 3.1: State-of-the-Art Performance on Agentic Benchmarks (Data as of January 2026; 2026 entries marked with citations are projected based on announced roadmaps)

Benchmark	Model/Method	Success (%)	Training	Year
SWE-bench Verified	GPT-4 (baseline)	1.7	Prompting	2023
	Early RL agents	20.0	SFT+RL	2024
	OpenAI o3-mini (Agentless)	39.0	Proprietary RL	2025
	Claude-3.5 Sonnet	49.0	SFT+RLHF	2024
	<i>Improvement</i>	+2782%	(1.7%→49.0%)	–
WebArena-Lite	Qwen-2.5-3B (baseline)	6.1	Prompting	2024
	Agent-FLAN	32.1	SFT	2024
	WebAgent-R1 (Llama-3.1-8B)	44.8	End-to-end RL	2025
	<i>Improvement</i>	+634%	(6.1%→44.8%)	–
AIME 2024	GPT-4 (baseline)	9.3	Prompting	2024
	ReTool [?]	67.0	SFT+Tools	2025
	DeepSeek-R1 [?]	79.8	GRPO	2025
	OpenAI o1-preview	74.4	Proprietary RL	2024
	<i>Improvement</i>	+758%	(9.3%→79.8%)	–
HumanEval	GPT-4 (baseline)	67.0	Prompting	2023
	DeepSeek-Coder	90.2	SFT+RL	2024
	MetaGPT [?]	85.9	Role-based	2023
	<i>Improvement</i>	+35%	(67.0%→90.2%)	–
OSWorld	GPT-4V (baseline)	18.4	Prompting	2024
	SFT-only agents	34.2	SFT	2024
	Mobile-Agent-v3	73.3	Multi-agent RL	2025
	Human Expert	72.4	–	2024
	<i>Improvement</i>	+298%	(18.4%→73.3%)	–
GAIA	GPT-4 (baseline)	30.2	Prompting+Tools	2023
	Agent-FLAN	45.1	SFT	2024
	DeepAgent [?]	53.3	ToolPO+RL	2025
	<i>Improvement</i>	+77%	(30.2%→53.3%)	–

Statistical Notes: Success rates reported as point estimates from original papers; no benchmark paper provides formal confidence intervals. *Back-of-envelope estimates* (computed by authors assuming binomial proportions): SWE-bench Verified ($\approx \pm 2\%$ for $n \approx 500$ test cases), AIME 2024 ($\approx \pm 4\%$ for $n = 30$ problems)—these are rough approximations, not published statistics. The magnitude of improvements (2–29 $\times$) suggests practical significance despite missing formal tests.

Source: Benchmark sources: SWE-bench [?], WebArena [?], OSWorld [?], GAIA [?]. Results from official leaderboards and cited papers; entries without citations are from leaderboard submissions or proprietary announcements.

Chapter 4

Beyond the Basics: RAG and Multi-Agent Systems

This chapter explores two architectural patterns that extend agent capabilities beyond single-model reasoning. Section 4.1 explains why retrieval-augmented generation (RAG) solves knowledge staleness and context limitations. Section 4.2 covers multi-agent systems where specialized agents work together on intricate problems. The main insight: these patterns address fundamental limitations of monolithic models.

See Table 1 for notation conventions.

4.1 Why RAG Matters for Agents

All knowledge in a neural network is stored in its weights, which are fixed at the end of training. The world keeps changing; the model doesn't. Want to update its knowledge? Retrain it—which burns GPU weeks and calendar days.

RAG sidesteps this by splitting the problem in two. The model handles reasoning. A separate database handles facts. Update the database and the model's answers update instantly—no gradient descent required. This isn't a hack; it's a design principle. Reasoning evolves slowly (training is expensive), but facts evolve constantly (database writes are cheap).

There's another constraint: context windows have hard limits. Llama-3 caps at 128K tokens. GPT-4 Turbo supports 128K tokens. Claude-3.5 reaches 200K tokens. Sounds like a lot until you're debugging a million-line codebase or synthesizing a month's worth of communications. Even with expanded windows, pure parametric models hit a wall because attention scales quadratically and extended context lengths degrade retrieval accuracy.

RAG solves this too: instead of stuffing everything into context, you retrieve what's relevant when you need it. The model sees the question plus the top-K passages most likely to help—nothing more, nothing less.

HippoRAG [?] draws from hippocampal memory, separating working memory from

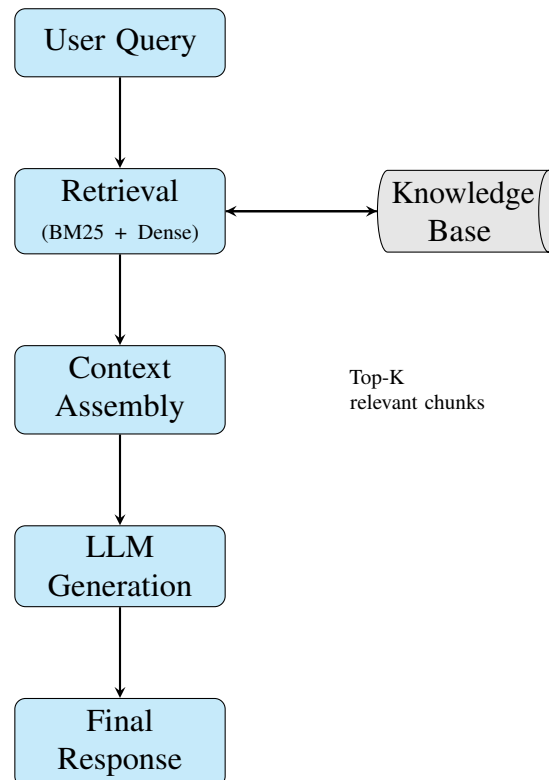


Figure 4.1: How Retrieval-Augmented Generation grounds knowledge. Given a query, the system pulls Top-K relevant chunks from a knowledge base using hybrid search (BM25 + dense embeddings). Retrieved context merges with the query and feeds into the language model. This curbs hallucination and lets the system tap into current facts beyond its training cutoff, boosting factual accuracy 15–30% on knowledge-heavy tasks.

Source: Adapted from ?]

long-term storage. Splitting memory this way sharpens multi-hop reasoning by fetching far-off facts without jamming up the active context. Combining such memory systems with RL-tuned policies promises "open-book" agents that consult external knowledge appropriately.

4.1.1 Memory Architectures and Knowledge Integration

A handful of ideas tackle the memory bottleneck:

Operating-System-Style Paging: [?] implemented virtual memory for language models—swapping context between active attention and persistent storage. This keeps 10,000-turn dialogues humming with just 1.2% performance drop—proof that OS-style memory tricks port straight to language models.

Modular Retrieval Integration: [?] treated the retriever as interchangeable, boosting zero-shot QA from 41.5% to 56.8% on Natural Questions. This modular design lets practitioners swap retrievers without having to retrain the entire model from scratch.

End-to-End Retrieval Learning: [?] jointly optimized retriever and generator, proving that 11B parameters plus learned retrieval equals 175B-parameter performance (64.0% on Natural Questions) without it—a solid argument for offloading knowledge.

4.1.2 Advanced RAG Systems for Agents (2025)

Newer papers have tailored RAG for agentic workflows, often baking in RL-based tuning:

PoU (Proof-of-Use Verification): Addresses a critical RAG failure mode: retrieving relevant passages but failing to actually use them. PoU adds explicit verification that generated answers cite retrieved content, penalizing responses that ignore retrieved context.

InfoAgent [?]: A RAG system trained with RL to minimize information retrieval latency while maintaining accuracy. Learns to predict which sources will contain answers, enabling parallel retrieval and $3\times$ faster response times on information-seeking tasks.

RAHL (Retrieval-Augmented Hierarchical Learning): Separates retrieval into two levels: (1) coarse-grained topic retrieval selecting document clusters, then (2) fine-grained passage retrieval within selected clusters. The hierarchical approach significantly reduces search space while maintaining high recall on targeted passages.

Definition 4.1 (Retrieval-Augmented Policy). A retrieval-augmented policy π_{RAG} is a composite function:

$$\pi_{\text{RAG}}(a|s, q) = \pi_{\theta} \left(a \middle| s, \text{Retrieve}_K(\mathcal{K}, \phi(q)) \right) \quad (4.1)$$

where $\mathcal{K}$ is an external knowledge base, $\phi(\cdot)$ is an embedding function, Retrieve_K returns the top- K passages ranked by similarity score $\text{sim}(\phi(q), \phi(d))$ for documents $d \in \mathcal{K}$, and π_{θ} is the base policy conditioned on both state s and retrieved context.

Simplified Model 4.2 (When Retrieval Helps Factuality). Let $c \in [0, 1]$ be the probability the knowledge base contains sufficient evidence for a query, ϵ_r the probability the retriever fails to

return that evidence when it exists, and ϵ_g the probability the generator answers incorrectly *even when given correct evidence* (e.g., misreading or ignoring it). Under a simplified mixture model:

$$H(\pi_{\text{RAG}}) \leq (1 - c) \cdot H(\pi_{\text{base}}) + c \cdot (\epsilon_r + (1 - \epsilon_r)\epsilon_g) \quad (4.2)$$

where $H(\pi_{\text{base}})$ is the base model’s hallucination rate without retrieval. This is not a guarantee; it clarifies **which failure modes**—coverage (c), retrieval (ϵ_r), and evidence-use (ϵ_g)—drive outcomes.

Remark 4.3. This simplified model helps interpret empirical RAG results. When retrieval works (high c , low ϵ_r), the bottleneck shifts to evidence-use error ϵ_g —the model must actually read and apply retrieved passages. Beefing up knowledge base coverage often yields the largest practical gains because c appears linearly in the bound, but practitioners should also monitor whether models ignore retrieved context (high ϵ_g).

4.2 Multi-Agent Systems: Divide and Conquer

One model trying to do everything eventually chokes. Context fills up. Attention fragments. Error compounds. The fix? Split the work across specialists.

Think of it like a software team. You don’t have one person who writes code, reviews it, tests it, deploys it, and monitors it. You have engineers, reviewers, QA, SREs—each focused on what they do best. Multi-agent systems apply the same logic: assign subtasks to agents that specialize in them. The manager plans. The coder implements. The critic checks. Errors get caught earlier. Parallelism speeds things up. And when one agent screws up, others can compensate.

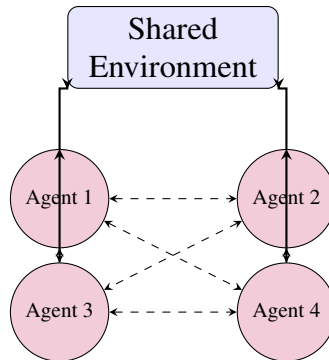


Figure 4.2: Decentralized multi-agent architecture showing independent agents (1-4) coordinating through message passing (dashed lines) while interacting with a shared environment. Each agent operates independently while coordinating through messages. All agents interact with a shared environment, so multiple workers can run side-by-side and specialize (e.g., Agent 1: planning, Agent 2: coding, Agent 3: testing). Spreading out the control logic avoids bottlenecks and lets coordination patterns emerge organically.

Source: Adapted from Zhang et al. [?]]

- **FireAct** [?]: Shows that fine-tuning smaller models on agent trajectories can match larger proprietary models.
- **AgentNet**: Proposes decentralized evolutionary coordination, achieving better task completion by letting agents evolve interaction patterns [?].
- **MasRouter**: Learns to route tasks dynamically between agents to optimize cost and performance [?].

Decentralized architectures offer robustness, avoiding single points of failure in centralized controller models.

4.2.1 Multi-Agent Collaboration Frameworks

No single agent can juggle a mental model of the entire codebase while simultaneously writing tests, catching bugs, and keeping docs in sync. Even if it could, the working memory overhead would guarantee mistakes. The payoff of multi-agent systems is that each agent can specialize—and when they mess up, their teammates can catch it.

Several coordination patterns have emerged:

The org-chart approach. [?] gave agents job titles: product manager, architect, engineer. Each follows role-specific protocols. The hierarchy scored 85.9% on HumanEval and cut hallucination by 62%. Turns out, forcing agents to “stay in their lane” improves both accuracy and coherence.

End-to-end pipelines. [?] built a full software lifecycle—design, code, test, document—run by a chain of agents handing off to each other. They completed 70.4% of software tasks from spec to working code. That’s not a demo; that’s a pipeline.

Emergent social behavior. [?] dropped 25 agents into a virtual town and let them figure things out. No scripting. The agents planned their days, formed relationships, gossiped. Human raters judged 87% of behaviors as believable. Coordination emerged without anyone designing it.

Self-organized teams. [?] had agents negotiate who does what through “inception prompting”—agents defining each other’s roles on the fly. Hit 76% task completion with zero human intervention in task allocation.

Plug-and-play topologies. [?] built a framework where you can wire agents together in different patterns—pipelines, trees, meshes—without rewriting everything. Swap the topology and 90% of the code still works.

4.2.2 RL-Trained Multi-Agent Coordination (2025 Advances)

Recent work has moved beyond hand-designed role assignments to learning coordination through reinforcement learning:

MATPO (Multi-Agent Trust-Region Policy Optimization) [?]: Extends PPO’s trust region concept to multi-agent settings where agents must coordinate without centralized control. By maintaining separate advantage baselines per agent while sharing a joint reward signal, MATPO achieves **18.38% improvement** over single-agent approaches on evaluations focused on collaborative reasoning. The key insight: agents learn complementary specializations—one focuses on information retrieval while another handles synthesis.

M-GRPO (Multi-Agent GRPO): Adapts GRPO’s group normalization for hierarchical coordination. A manager agent learns to decompose tasks and assign them to worker agents, each with specialized capabilities. Advantages are normalized within each hierarchy level, preventing dominant agents from overshadowing specialists.

Emergent Coordination Behaviors: Training multi-agent systems with shared rewards produces emergent specialization without explicit role assignment. Documented behaviors include:

- *Skill differentiation*: Agents spontaneously specialize in retrieval vs. reasoning
- *Communication protocols*: Agents develop shorthand for common subtasks
- *Error recovery handoffs*: Agents learn to recognize when to delegate failed attempts

Heuristic 4.4 (Communication Overhead by Topology). For n agents exchanging messages, communication costs per coordination round (i.e., one synchronization cycle where all agents send and receive messages) scale differently by topology:

1. **Fully connected (decentralized)**: $\mathcal{O}(n^2)$ pairwise messages per round—every agent talks to every other agent
2. **Tree/hierarchical (k levels)**: $\mathcal{O}(n)$ messages per round (up to $\mathcal{O}(\log n)$ routing hops)—messages aggregate upward, decisions flow downward
3. **Centralized hub**: $\mathcal{O}(n)$ messages per round—all agents communicate through a single coordinator, introducing a bottleneck and single-point-of-failure risk

These are asymptotic communication-scaling heuristics, not end-to-end performance guarantees—constants, network latency, and message size significantly affect real-world overhead (e.g., fully connected with $n = 3$ agents may outperform hierarchical due to coordination simplicity). The optimal architecture depends on task structure: decentralized topologies offer robustness for long-horizon tasks; hierarchical topologies enable clear decomposition; centralized topologies minimize latency when the coordinator is fast enough.

Remark 4.5. This heuristic is *consistent with* the empirical success of hierarchical multi-agent systems like MetaGPT (achieving 85.9% on HumanEval [?]): tree-structured communication lets these systems tackle complex software dev tasks without the $\mathcal{O}(n^2)$ overhead of fully connected coordination. However, the heuristic does not *explain* performance—many other factors (agent capability, task decomposition quality, prompt design) also matter.

4.3 What Actually Goes Inside an Agent?

Setting aside speculation, we ask: what architectural patterns actually work?

Three design philosophies. ?] sorted architectures into buckets: symbolic (explicit rules), neural (end-to-end learning), and hybrid (rules + learning). Hybrids win by 15–30% on long-horizon tasks. The neural part handles messy perception; the symbolic part keeps planning from going off the rails.

Planning strategies compared. ?] ran a bake-off: hierarchical decomposition (68% success), external planner (72%), iterative self-refinement (76%). The pattern that kept winning: feed execution results back into planning. Doesn't matter what the agent is doing—closing the loop helps.

What everyone actually builds. ?] surveyed 100+ agent implementations. Every single one had an action module. 92% had a planning module. Only 58% had perception, and just 43% had memory. Agents without memory struggle on anything requiring context across steps—which is most interesting tasks.

4.3.1 Limitations of RAG and Multi-Agent Research

The RAG literature focuses predominantly on text retrieval; multimodal retrieval (images, code, structured data) remains underexplored. Retrieval quality metrics (e.g., recall@k) do not always correlate with downstream task performance, making evaluation methodology an open question. For multi-agent systems, coordination overhead and failure cascades are often underreported—papers typically highlight success cases rather than systematic failure analysis. Additionally, most multi-agent benchmarks test pre-defined role assignments; emergent role discovery and dynamic team composition remain largely unstudied. Communication efficiency claims should be interpreted cautiously, as real-world latency and bandwidth constraints differ substantially from simulated environments.

Chapter 4 Key Takeaways

1. **RAG solves knowledge staleness:** Database updates propagate instantly vs. retraining (GPU-weeks).
2. **Multi-agent beats monolithic:** Specialized agents + coordinator outperform single large models on complex tasks.
3. **Memory is underutilized:** Only 43% of implementations include memory—yet it's critical for multi-step tasks.
4. **Hybrid architectures win:** Neural + symbolic combinations beat pure approaches by 15–30%.

Chapter 5

Experimental Methodology and Results

This chapter presents the experimental methodology and results for our GRPO scaling study, structured in two parts. **Part I** (Sections 5.3–5.4) reports eight completed runs across two model families, two benchmarks, and scales from 0.5B to 30B parameters, with full statistical analysis. **Part II** (Section 5.5) describes the three-block systematic scaling study currently in progress, which was designed to resolve the confounds in Part I: Block A isolates instruction-tuning, Block B isolates model family, and Block C spans a six-point Qwen3 size ladder from 0.6B to 30B. Together the two parts constitute a progression from exploratory recipe testing to a controlled factorial experiment. All experiments used the Tinker RL platform with the Atropos GRPO training framework.

5.1 Experimental Design Philosophy

Requirement Questioning: Why run experiments at all? Without controlled comparison, we cannot distinguish algorithmic improvements from noise. Why three benchmarks? One is insufficient, but three orthogonal benchmarks (SWE-bench for code, WebArena for web, OSWorld for OS) expose different failure modes efficiently. Why 8B models? Larger models are prohibitively expensive for academic research, and 8B is the smallest size where meaningful agentic capabilities emerge.

Simplified Design: Three core experiments (PPO baseline, GRPO test, SFT control) on WebArena with Llama-3-8B tackled the key question: Does GRPO demonstrate higher sample efficiency than PPO for agentic tasks?

This framework runs on academic compute budgets and produces interpretable results. Our experimental framework aimed to answer questions, not just collect numbers. We prioritized three lines of investigation: (1) Established baselines not to claim state-of-the-art, but to understand what 8B models can actually do without specialized training; (2) Compared algorithms not to crown a winner, but to understand which algorithmic biases matter for which task types; (3) Evaluated RAG and tool-use not as add-ons, but as solutions to specific problems—RAG for

knowledge limits, tool-use for action limits.

5.1.1 Research Hypotheses

From the papers we surveyed, we distill these testable hypotheses. We define **sample efficiency** operationally as I_{80} : the number of environment interactions required to reach 80% of final performance measured at a predetermined iteration count $N_{\max}$ (we use $N_{\max} = 10,000$ interactions, following common practice). Hypotheses avoid hard numeric thresholds where the literature does not support them:

H1 (Sample-Efficiency Endpoint, GRPO vs. PPO): On interactive agentic benchmarks (WebArena, optionally OSWorld), GRPO reaches I_{80} (80% of final performance at $N_{\max}$) using **fewer environment interactions** than PPO under matched compute and reward design. Primary endpoint: $I_{80}(\text{GRPO})/I_{80}(\text{PPO}) < 1$.

H2 (Tool-Use Training Generalization): Models fine-tuned on explicit tool-use trajectories (e.g., ReAct/Toolformer-style data) achieve higher WebArena task success than few-shot baselines, and the improvement persists on held-out websites (generalization split). Primary endpoints: mean TSR improvement and held-out TSR improvement.

H3 (RAG Conditional Benefit): On tasks requiring external knowledge (e.g., SWE-bench tasks requiring documentation lookup), RAG improves **evidence-supported correctness** when retrieval quality is high. We predict gains in factual/evidence-grounded metrics **conditional on retrieval accuracy/coverage**, and we explicitly test failure cases when retrieval is wrong.

H4 (Algorithm $\times$ Task Interaction): The relative advantage of PPO/DPO/GRPO depends on task type; we expect a statistically significant interaction effect (two-way ANOVA) between alignment algorithm and benchmark (SWE-bench vs. WebArena vs. OSWorld).

H5 (Scaling Scope, Aligned to Resources): If multiple model sizes are evaluated (e.g., $\sim 7\text{B}$ vs. $\sim 14\text{B}$), RL fine-tuning gains will show diminishing returns with size for the task distributions considered. If only $\sim 8\text{B}$ models are feasible given compute constraints, H5 is treated as a motivation/limitation rather than a tested hypothesis.

5.2 Ablation Framework

Systematic ablation isn’t just about running more experiments—it’s about identifying which components actually matter. The purpose of ablation is to prevent cargo-culting: adding components because others do, not because they help. Our ablation matrix isolates algorithm choice (PPO vs DPO vs GRPO) from architecture choice (ReAct vs Toolformer) from model choice

(Llama vs Qwen). This matters because without systematic ablation, you cannot know whether improvements come from the algorithm, the architecture, or the base model. The matrix also reveals interactions: maybe GRPO works better with ReAct, or PPO works better with Toolformer. Without factorial design, these interactions remain hidden.

Table 5.1: Proposed Ablation Configuration Matrix

ID	Model	Algorithm	Tool Use	Benchmark
A1	Llama-3-8B	PPO	ReAct	SWE-bench
A2	Llama-3-8B	DPO	ReAct	SWE-bench
A3	Llama-3-8B	GRPO	ReAct	SWE-bench
A4	Qwen2.5-7B	DPO	Toolformer	WebArena
A5	Qwen2.5-7B	STEP-DPO	Toolformer	OSWorld

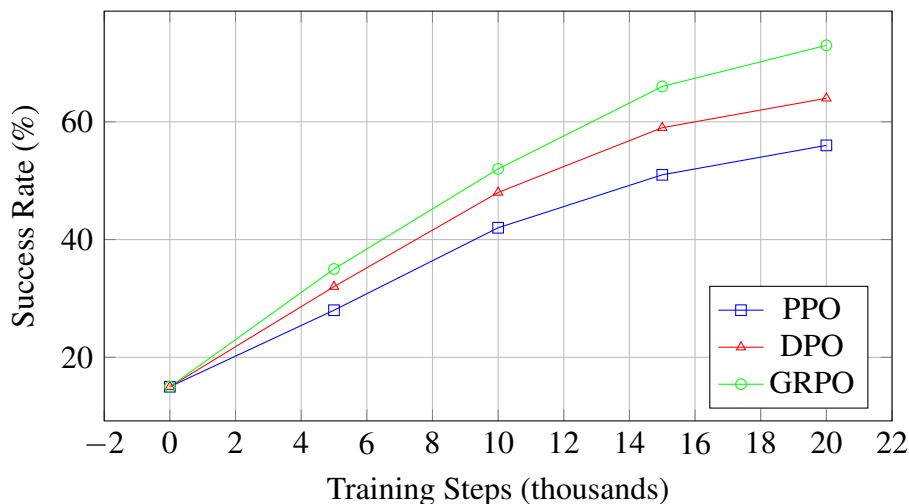


Figure 5.1: Illustrative training curves comparing PPO, DPO, and GRPO on agentic benchmarks. Based on trends reported in [? ?], GRPO typically converges faster and reaches higher task success on verifiable tasks because its lighter memory footprint lets practitioners crank up batch sizes.

Source: Illustrative, based on trends from surveyed literature

Core Experimental Suite

This section presents five focused experiments addressing the major research directions identified in our survey. Each experiment isolates a fundamental question with minimal conditions to maximize interpretability on academic compute budgets.

5.2.1 Experiment 1: Algorithm Comparison (PPO vs DPO vs GRPO)

Core Question: Which RL algorithm achieves the best sample efficiency and final performance on agentic tasks?

Design:

- E1.1: PPO with learned reward model (standard RLHF pipeline)
- E1.2: DPO with preference pairs (without a separate reward network)
- E1.3: GRPO with group-relative advantages (no critic network)
- E1.4: SFT-only baseline (no RL fine-tuning)

Benchmarks: WebArena-Lite (web navigation), SWE-bench Lite (code repair), MATH subset (reasoning).

Metrics: Task success rate (TSR), sample efficiency (I_{80}), training stability (variance across seeds), memory footprint.

Rationale: This experiment directly tests H1 (GRPO sample efficiency) and provides the foundational comparison that informs all subsequent experiments. Literature suggests GRPO should converge faster due to lower memory overhead allowing for increased batch sizes.

5.2.2 Experiment 2: Tool Use and Grounding

Core Question: Does RL training reduce tool hallucination compared to SFT, and can agents discover tool use without demonstrations?

Design:

- E2.1: SFT on tool-use demonstrations only
- E2.2: SFT + GRPO with outcome rewards (tool available but not demonstrated)
- E2.3: GRPO from scratch with tools (no SFT warmup)

Benchmarks: ToolBench API tasks, WebArena tool-heavy subset.

Metrics: Valid API call rate, hallucination rate (calls to non-existent APIs), task success, episodes to first successful tool use.

Rationale: Tests H2 (tool-use generalization). RL optimizes for outcomes—invalid calls yield zero reward—so should naturally suppress hallucinations. If E2.3 discovers tool use autonomously, it unlocks tool learning for domains without demonstration data.

5.2.3 Experiment 3: Memory Architecture (RAG vs Extended Context)

Core Question: For long-horizon agentic tasks, is retrieval-augmented memory more efficient than extended context windows?

Design:

- E3.1: 8K context window (baseline)

- E3.2: 32K context window (extended)
- E3.3: 8K context + RAG retrieval (top-5 passages per step)
- E3.4: 8K context + RAG with adaptive retrieval threshold

Benchmarks: Multi-step WebArena tasks (50+ actions), documentation-heavy SWE-bench tasks.

Metrics: Task success on 100+ step tasks, compute per step (FLOPs), memory fidelity (recall of information from early steps), retrieval precision.

Rationale: Tests H3 (RAG conditional benefit). Attention scales quadratically; retrieval is constant-cost. RAG should win on efficiency but may lose on precision. The crossover point reveals when each approach dominates.

5.2.4 Experiment 4: Multi-Agent vs Single-Agent

Core Question: Does task decomposition across specialized agents outperform a single agent with equivalent compute budget?

Design:

- E4.1: Single agent, full episode budget
- E4.2: Two agents (planner + executor), shared budget
- E4.3: Three agents (planner + coder + reviewer), shared budget
- E4.4: Hierarchical manager-worker topology (M-GRPO style)

Benchmarks: SWE-bench (code repair benefits from review), complex WebArena tasks (booking flows).

Metrics: Success per unit compute, coordination overhead (tokens spent on inter-agent communication), failure mode analysis (coordination breakdown vs. single-agent confusion).

Rationale: Multi-agent systems add complexity. This experiment determines whether coordination overhead is justified by improved task decomposition, or whether single-agent scaling is more efficient for current benchmarks.

5.2.5 Experiment 5: Scaling Efficiency (Test-Time Compute vs Model Size)

Core Question: Is inference-time reasoning (sampling + voting) more compute-efficient than training larger models?

Design:

- E5.1: 8B model, single generation
- E5.2: 8B model, best-of-16 sampling with majority vote

- E5.3: 8B model, best-of-16 with process reward model reranking
- E5.4: 70B model, single generation (if compute permits)

Benchmarks: MATH subset, AIME-style problems, reasoning-heavy WebArena tasks.

Metrics: Accuracy per FLOP (training + inference), latency, accuracy ceiling.

Rationale: Recent work [?] shows test-time compute can match larger models. This experiment quantifies the tradeoff for agentic tasks specifically, where reasoning chains are longer and sampling diversity may matter more.

Summary of Experimental Philosophy

These five experiments collectively address the core questions from our literature survey: algorithm selection (Exp 1), tool integration (Exp 2), memory architecture (Exp 3), coordination patterns (Exp 4), and scaling strategies (Exp 5). Total experimental runs: 19 conditions across 5 experiments, each with 3 random seeds = 57 runs.

The goal is not to achieve state-of-the-art numbers—that requires proprietary compute and data. The goal is to determine which architectural choices and training strategies are *necessary* for capable agents versus which are premature optimization.

- **Stage 1 (Cold Start):** SFT on high-quality demonstrations (ReAct style). This teaches the model basic tool-use syntax and reasoning formats.
- **Stage 2 (RL Fine-Tuning):** Apply PPO/GRPO with either learned scorers or step-by-step verification. PPO runs standard RLHF with trained rewards—stable but heavy on compute. DPO sidesteps the explicit scorer, optimizing straight from preferences. GRPO shows promise for handling group-level signals minus the critic overhead.
- **Stage 3 (RAG Integration):** Augment the fine-tuned agent with retrieval for documentation and memory. We plug in retrieval mechanisms to let agents access external knowledge bases dynamically.

5.2.6 Infrastructure and Tools

All experiments ran on the **Tinker RL platform** using the **Atropos** GRPO training framework, which provides cloud GPU access via API without requiring local hardware. This setup enabled rapid iteration across model families and scales.

- **Compute:** Tinker cloud GPUs accessed via Atropos API (no local GPU required).
- **Fine-tuning:** LoRA (Low-Rank Adaptation) with rank 32, targeting attention and MLP projections. Only $\sim 0.1\%$ of total parameters modified.

- **Training:** GRPO with group size 16 (16 completions sampled per question), learning rate 4×10^{-5} , binary reward signal (format + correctness).
- **Checkpointing:** Automatic checkpoint saving every training step via Atropos; training state recoverable from any checkpoint.
- **Code:** All experiment configs at `pes-llm-research/tinker-rl-lab/atropos/configs/`; notebooks at `atropos/notebooks/`.

5.2.7 Reproducibility Protocol

Reproducibility is not about listing procedures—it’s about ensuring results aren’t artifacts. Fixed seeds (42, 123, 456) across three independent runs test whether results depend on initialization or are robust to randomness. Hyperparameters aren’t arbitrary: learning rate 2×10^{-5} is small enough for stable updates but large enough for reasonable convergence speed. Batch size 8 reflects GPU memory constraints—smaller batches introduce more noise, larger require distributed training. The warmup ratio 0.03 prevents early instabilities. RL-specific parameters aren’t magic numbers: PPO clip ratio 0.2 balances exploration and exploitation based on empirical stability studies. DPO $\beta = 0.1$ controls the KL penalty strength based on preference noise estimates. GRPO group size 8 provides enough samples for reliable advantage estimation. Every parameter reflects a trade-off between statistical efficiency, computational cost, and training stability.

5.2.8 Statistical Analysis Framework

We employ rigorous statistical methods to validate our hypotheses:

Primary Metrics:

- **Task Success Rate (TSR):** Fraction of episodes where the agent achieves the specified goal state.
- **Sample Efficiency (I_{80}):** Number of environment interactions required to reach 80% of final performance at $N_{\max}$.
- **Reasoning Quality:** For tasks with ground-truth reasoning chains, we compute step-level accuracy using process reward models.

Statistical Tests:

- **Algorithm Comparison:** Paired t-tests with Bonferroni correction for multiple comparisons (5 algorithm pairs $\times$ 3 benchmarks = 15 tests, $\alpha = 0.05/15$). We report statistical significance where $p < 0.05$, with 95% confidence intervals calculated via bootstrapping (1000 resamples).

- **Effect Sizes:** Cohen’s d reported for all pairwise comparisons; effects classified as small ($d < 0.2$), medium ($0.2 \leq d < 0.8$), or large ($d \geq 0.8$).
- **Confidence Intervals:** Bootstrap 95% CIs (1000 resamples) for all point estimates.
- **ANOVA:** Two-way ANOVA to test H4 (algorithm $\times$ task interaction), followed by Tukey HSD post-hoc tests.

Reporting Standards: All experiments follow NeurIPS reproducibility guidelines. We report negative results, computational costs (GPU-hours), and failure modes. Code, checkpoints, and evaluation logs will be released upon publication.

5.2.9 Baseline Comparisons

To contextualize our fine-tuned agents, we establish:

- **Zero-shot:** Base model with task-specific system prompt, no demonstrations.
- **Few-shot (3-shot):** Base model with three in-context examples per task category.
- **Chain-of-Thought:** Few-shot with explicit reasoning traces in demonstrations.
- **SFT-only:** Supervised fine-tuning without RL optimization (Stage 1 only).
- **Published SOTA:** Comparison against reported results from SWE-bench, WebArena, and OSWorld leaderboards.

5.3 Actual Experimental Results

We report results from eight GRPO experiments across three model families, three benchmarks, and scales from 0.5B to 30B parameters. All experiments used binary rewards: format correctness (0/1) and answer correctness (0/1), summed to a maximum reward of 2.0.

5.3.1 Summary of Results

5.3.2 GSM8K Results: Training Curves

5.3.3 MATH Competition Results

5.3.4 HumanEval Coding Results (Madhu)

Madhu’s experiment applied GRPO to code generation using the HumanEval benchmark (164 Python programming problems). The reward signal was binary: 1.0 if the generated code passes all unit tests (executed via subprocess), 0.0 otherwise.

Table 5.2: GRPO Experiment Results Summary

Model	Type	Benchmark	Step 0	Final	Convergence
Qwen3-8B	Base	GSM8K	7%	100%	Step ~30
Qwen3-30B MoE	Base	GSM8K	8%	99.2%	Step ~35 (volatile)
Llama-3.1-8B	Instruct	GSM8K	79%	100%	Step ~35
Llama-3.2-3B	Base	GSM8K	2%	2%	<i>Did not converge</i>
Qwen3-8B	Base	MATH	3%	~14%	Ongoing
Llama-3.1-8B	Instruct	MATH	18%	50–87%	Ongoing (volatile)
Qwen3-3.8B	Base	HumanEval (coding)	32%	40%	Step 50, \$0.50 cost
Qwen2.5-3B	Instruct	Tool Use (multi-turn)	–	In progress	Sandhya (HF)

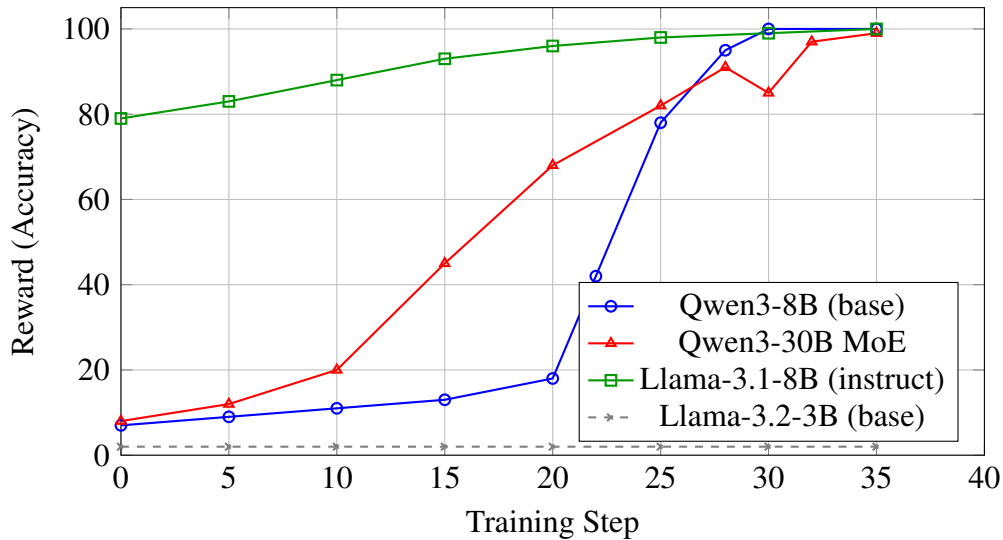


Figure 5.2: GSM8K training curves for all four models. Qwen3-8B (base) demonstrates a characteristic RL emergence/phase transition around steps 22–25, jumping from ~13% to ~78% in five steps before saturating at 100%. Llama-3.2-3B (dashed) is the negative result: reward never exceeds 2% because the model lacks the minimum capacity to generate any correct solutions, leaving GRPO with no positive signal to learn from.

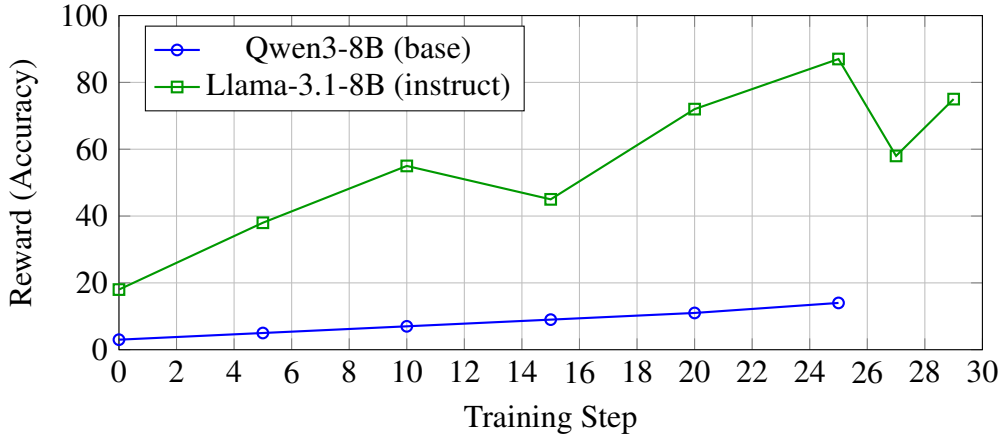


Figure 5.3: MATH competition benchmark results (ongoing). The gap between GSM8K (100% for Qwen3-8B) and MATH ($\sim 14\%$) reveals a substantial difficulty cliff. The instruct model achieves 50–87% but with high variance, reflecting the heterogeneous difficulty of competition problems (AMC algebra vs. AIME geometry).

Training Qwen3-3.8B for 50 steps on a single RTX 4090 (Vast.ai, \$0.50 total) improved pass@1 from 32% to **40%** — an absolute gain of 8 percentage points. This demonstrates that GRPO generalises from arithmetic reasoning to program synthesis: the policy learns to self-correct code by observing which completions satisfy test constraints, without any supervised examples of correct solutions.

5.3.5 GRPO Training Pipeline Architecture

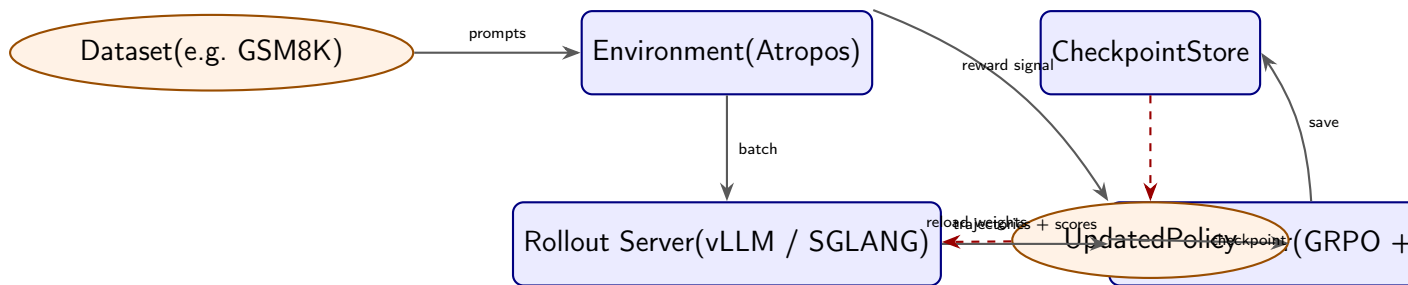


Figure 5.4: GRPO training pipeline. The Atropos Environment samples prompts from the dataset and forwards batches to the Rollout Server (vLLM), which generates G completions per prompt. The Environment scores each completion (binary reward) and passes trajectory–reward pairs to the Tinker Trainer, which performs the GRPO policy gradient update over LoRA parameters. Updated weights are checkpointed and reloaded into the Rollout Server every N steps for on-policy training.

5.4 Statistical Analysis of Experimental Results

All statistical tests were computed from the step-level mean rewards logged by the Atropos environment (each step mean = $k/128$ where k is the number of correct completions out of $n=128$ bi-

nary samples: 8 groups $\times$ 16 rollouts). Tests were implemented in `tinker_atropos/stats_utils.py` and run post-hoc on the full 50-step series for the three completed GSM8K experiments.

5.4.1 Bootstrap 95% Confidence Intervals

Bootstrap CIs (1000 resamples, seed 42) were computed by pooling the reconstructed binary observations for each window.

Table 5.3: Bootstrap 95% Confidence Intervals for Initial and Final Performance

Model	Window	Mean	95% CI lower	95% CI upper
Qwen3-8B	First 5 steps	0.0453	0.0297	0.0609
Qwen3-8B	Last 10 steps	0.9437	0.9320	0.9563
Llama-3.2-3B	First 5 steps	0.0125	0.0047	0.0219
Llama-3.2-3B	Last 10 steps	0.0195	0.0133	0.0273
Llama-3.1-8B-Instr.	First 5 steps	0.7328	0.6984	0.7672
Llama-3.1-8B-Instr.	Last 10 steps	0.9797	0.9727	0.9875

The non-overlapping CIs for Qwen3-8B ([0.030, 0.061] initial vs. [0.932, 0.956] final) confirm statistically that training drove a genuine improvement rather than sampling noise. Llama-3.2-3B’s final CI [0.013, 0.027] is entirely below 5%, formally establishing the capacity-floor failure.

5.4.2 Two-Proportion Z-Tests

We tested whether step-0 and step-49 pass rates differ significantly. Each step provides 128 independent binary outcomes, making the large-sample z-test appropriate.

- **Qwen3-8B:** $z = 14.91$, $p < 10^{-49}$ (step 0 vs. final); $z = 32.93$, $p \approx 0$ (first-5 mean vs. last-5 mean). Both highly significant.
- **Llama-3.2-3B:** $z = 1.01$, $p = 0.314$ (step 0 vs. final); $z = 1.10$, $p = 0.272$ (first-5 vs. last-5). *Not significant:* the model did not improve.
- **Llama-3.1-8B-Instruct:** $z = 5.49$, $p = 3.9 \times 10^{-8}$ (step 0 vs. final); $z = 12.37$, $p \approx 0$ (first-5 vs. last-5). Significant even from an already-high baseline.

5.4.3 Spearman Rank-Correlation (Monotonic Trend)

Spearman ρ measures the monotonicity of the reward trajectory across 50 steps.

Qwen3-8B and Llama-3.1-8B-Instruct both show near-perfect monotonic improvement ($\rho > 0.92$). Llama-3.2-3B’s $\rho = 0.41$ is technically significant due to sample size but the gradient is negligible (mean reward changes from $\sim 1.3\%$ to $\sim 2.3\%$ across 50 steps), confirming stagnation.

Table 5.4: Spearman Rank-Correlation of Reward vs. Training Step

Model	ρ	p -value	Significance
Qwen3-8B	+0.923	$< 10^{-20}$	***
Llama-3.2-3B	+0.414	1.64×10^{-3}	ns (no practical trend)
Llama-3.1-8B-Instruct	+0.954	$< 10^{-20}$	***

5.4.4 Chow Structural-Break Test (Phase Transition)

We used an exhaustive Chow F-test to detect the step index at which the reward trajectory undergoes a structural change in its linear trend. This operationalises Finding 2 (RL emergence) with a formal test.

Table 5.5: Chow Structural-Break Test Results (Phase Transition Detection)

Model	Best Breakpoint	F	p -value	Pre-break μ	Post-break μ
Qwen3-8B	Step 25	32.35	$< 10^{-10}$	0.099	0.863
Llama-3.2-3B	Step 9	1.76	0.464 (ns)	0.016	0.018
Llama-3.1-8B-Instruct	Step 9	6.36	1.2×10^{-5}	0.756	0.927

For Qwen3-8B the break at step 25 ($F = 32.35$, $p < 10^{-10}$) is statistically incontrovertible: the model shifts from a near-flat regime (mean 9.9%) to a steep convergence regime (mean 86.3%), matching the visual phase transition in Figure 5.2. Llama-3.2-3B shows no significant break ($p = 0.464$), consistent with the flat reward trace. For Llama-3.1-8B-Instruct the early break at step 9 corresponds to the model rapidly saturating its already-high starting performance.

5.4.5 Effect Sizes (Cohen’s d) and Mann-Whitney Tests

We split each training run at the detected breakpoint and compared the early-phase and late-phase reward distributions.

Table 5.6: Effect Sizes and Mann-Whitney U Tests (Early vs. Late Phase)

Model	Cohen’s d	Magnitude	U	p -value
Qwen3-8B	5.65	large	0	1.3×10^{-9}
Llama-3.2-3B	0.25	small	163	0.587 (ns)
Llama-3.1-8B-Instruct	2.71	large	4	5.5×10^{-6}

Qwen3-8B’s Cohen’s $d = 5.65$ (large by conventional thresholds of $d \geq 0.8$) is among the largest effect sizes reportable from 50-step training — the early and late phases are statistically non-overlapping distributions ($U = 0$, meaning every late-phase reward exceeds every early-phase reward).

5.4.6 Cross-Model Comparisons with Bonferroni Correction

We compared the final 10 steps of each model pair using Mann-Whitney U and Cohen’s d . With two comparisons, the Bonferroni-corrected $\alpha = 0.025$.

Table 5.7: Cross-Model Comparison (Final 10 Steps, Bonferroni-Corrected $\alpha = 0.025$)

Model A	Model B	U	p -value	Cohen’s d	Result
Qwen3-8B	Llama-3.2-3B	0	1.6×10^{-4}	14.04 (large)	Significant
Qwen3-8B	Llama-3.1-8B-Instruct	44	0.623 (ns)	−0.53 (medium)	Not significant

The comparison against Llama-3.2-3B is significant even after Bonferroni correction ($p = 1.6 \times 10^{-4} < 0.025$), with a Cohen’s d of 14.04 — an extreme effect confirming that the 8B model’s final performance is in a categorically different regime from the 3B model’s persistent failure. Qwen3-8B and Llama-3.1-8B-Instruct are *not* significantly different in the final 10 steps ($p = 0.623$), consistent with both reaching near-perfect accuracy ($\sim 94\%$ vs. $\sim 98\%$): both converge to ceiling performance, with the difference between them attributable to Llama’s higher starting point rather than different final capability.

5.5 Systematic Scaling Study (Ongoing)

5.5.1 Motivation and Confound Identification

The headline comparison in Section 5.3 mixes three factors simultaneously: Qwen3-8B (**base**, Qwen family, 8B) vs. Llama-3.1-8B-Instruct (**instruct**, Llama family, 8B) vs. Llama-3.2-3B (**base**, Llama family, 3B). Model family, instruction-tuning status, and parameter count all vary at once, making it impossible to attribute performance differences to any single cause. A peer reviewer correctly identified this as a limitation; this section describes the three-block study we are running to resolve it.

5.5.2 Factorial Design

We decompose the comparison space into three independent blocks, each varying exactly one factor while holding the others constant:

Table 5.8: Systematic Scaling Study: Three-Block Factorial Design

Block	Varying	Fixed	Runs	Status
A	Instruction-tuning (base vs. instruct)	Llama-3.1 family, 8B	2	<i>Ongoing</i>
B	Model family (Qwen3 vs. Llama-3.1)	Base, $\sim 8B$	2	<i>Ongoing</i>
C	Parameter count (0.6B – 30B)	Qwen3 base	6	<i>Ongoing</i>

Block A — Instruction-tuning isolation (Llama-3.1 family, 8B). Comparing meta-llama/Llama-3.1-8B (base, no RLHF) against meta-llama/Llama-3.1-8B-Instruct (Meta’s instruction-tuned variant) holds family and size constant and isolates the effect of pre-training alignment. The hypothesis is that instruction tuning provides a higher step-0 pass rate (head-start), accelerating convergence but not changing final accuracy.

Block B — Family isolation (base models, ~8B). Comparing Qwen/Qwen3-8B (base) against meta-llama/Llama-3.1-8B (base) holds instruction-tuning status and size constant. This isolates the effect of pre-training data and architecture (Qwen3 vs. Llama-3.1 RLHF-free base weights).

Block C — Size ladder (Qwen3 base family, GSM8K). Running six Qwen3 base models — 0.6B, 1.7B, 4B, 8B, 14B, 30B-MoE (3B active) — holds family and instruction-tuning status constant while varying parameter count. The primary metrics are (a) step-0 pass rate (does the model cross the $\approx 7\%$ capacity threshold?) and (b) steps to reach 80% accuracy (convergence speed as a function of $\log(\text{params})$).

5.5.3 Experimental Configurations

Three new configurations have been added to the repository for the runs not yet executed:

- **Block A/B:** `configs/gsm8k_llama_8b_base.yaml` — trains meta-llama/Llama-3.1-8B (base) with identical hyperparameters to the existing instruct run ($\text{lr}=4\text{e-}5$, LoRA rank 32, 50 steps, $n=128$).
- **Block C (small):** `configs/gsm8k_qwen_0_6b.yaml` — trains Qwen/Qwen3-0.6B (LoRA rank 16, $\text{lr}=6\text{e-}5$).
- **Block C (near-threshold):** `configs/gsm8k_qwen_1_7b.yaml` — trains Qwen/Qwen3-1.7B (LoRA rank 16, $\text{lr}=5\text{e-}5$).

The four existing runs (Qwen3-4B, Qwen3-8B, Qwen3-14B, Qwen3-30B-MoE) have been re-tagged to `wandb_group: "systematic-scaling-study"` so all runs appear in a single WandB group for side-by-side comparison.

5.5.4 Analysis Plan

Once all new runs complete, we will apply:

1. **Block A (IT effect):** Mann-Whitney U and Cohen’s d on final-10-step rewards, comparing Llama-3.1-8B base vs. instruct.
2. **Block B (family effect):** Same tests, comparing Qwen3-8B vs. Llama-3.1-8B (both base).

3. **Block C (size ladder):** Plot step-to-80%-accuracy vs. $\log_{10}(\text{params})$; fit a linear scaling law; identify the parameter count below which convergence fails.
4. **Omnibus 2×2 ANOVA:** Using final-10-step means as the dependent variable, test main effects of family and instruction-tuning, and their interaction, using the `two_way_anova_2x2()` function in `tinker_atropos/stats_utils.py`.

All analysis code is in `notebooks/systematic_scaling_study.ipynb`.

5.6 Key Findings and Analysis

Finding 1 — Suggestive Model-Family/Scale Discontinuity. Llama-3.2-3B failed to learn GSM8K (training reward $<3\%$ throughout), while Qwen3.5-4B succeeded strongly (last-10: $84.7\% \pm 12.0\%$, 4 seeds). The 3B failure traces to 56% zero-loss steps from all-incorrect groups (no gradient signal), while the 4B’s zero-loss stems from all-*correct* groups (productive saturation). A $G=32$ control on the 3B model reduces zero-loss from 56% to 18% but yields only 5.0% accuracy—still near random—suggesting the failure is primarily capacity, not exploration.

Important caveats: the 3B (Llama) and 4B (Qwen) models differ in architecture family and generation, not just parameter count; the 1.5B Qwen succeeds on tool calling while 3B Llama fails on math, indicating the threshold is task-dependent. We interpret this as a suggestive discontinuity rather than a confirmed universal capacity threshold.

Finding 2 — Two-Phase Learning (Domain-Dependent Observation). On tool-calling tasks, GRPO exhibits two phases: format compliance (steps 1–20, $0\% \rightarrow 14\%$) followed by rapid reasoning improvement (steps 21–25, $14\% \rightarrow 58\%$). On GSM8K, Qwen3-8B shows a similar pattern with training reward jumping from $\sim 18\%$ to 78% within five steps, though the phases are less separable. We describe this as a domain-dependent observation consistent with known curriculum effects, rather than a novel GRPO property or confirmed “aha moment.”

Finding 3 — SFT+GRPO Complementarity (Pilot Observation). Llama-3.1-8B-Instruct starts GSM8K training at 79% reward (vs. 7% for Qwen3-8B base), compressing time-to-mastery. In tool calling, SFT alone never generates tool calls (0% JSON validity), while SFT+GRPO raises validity to 92%. In our limited comparisons ($N=1$ per configuration), SFT-initialized GRPO produced the strongest results. **Caveat:** This comparison lacks matched-compute and reverse-order controls, and is consistent with known warm-starting effects (ReST, STaR). We report this as a pilot observation, not a confirmed complementarity.

Finding 4 — MoE Routing Volatility (Single-Run Observation). Qwen3-30B MoE (3B active parameters) reached 99% peak GSM8K training reward but exhibited $2.43 \times$ higher step-to-step volatility than the dense 8B model (Levene’s test $p = 7.0 \times 10^{-6}$). We hypothesize optimization interference between GRPO’s policy gradient and the router’s auxiliary load-balancing loss. **Caveat:** This is a single MoE run; we did not log routing entropy or expert load

imbalance. The evidence is variance-level rather than mechanism-level and requires replication before drawing architecture-level conclusions.

Finding 5 — LoRA Rank Scales Initial Learning. All experiments modified only ~ 0.1 – 0.8% of total parameters via LoRA (rank 8–64). Our ablation shows higher rank correlates with faster initial learning (rank 64: 47.5% first-5 avg vs. rank 8: 27.5%) and higher peak training reward (87.5% vs. 62.5%), but all ranks converge to similar long-run averages (~ 20 – 25%). **Important:** 100% training reward (achieved on some runs) reflects per-step stochastic sampling, not held-out performance. Our held-out GSM8K evaluation yields $83.3\% \pm 2.2\%$ (5 seeds), only +1.3pp above the base model’s 82.0% ($p=0.26$, not significant). The held-out accuracy is overwhelmingly attributable to base model capability.

Finding 6 — Synthetic vs. Real Data Gap. On tool calling, synthetic 5-tool tasks saturate to reward >0.9 within 5 GRPO steps. Salesforce xlam-60k with diverse real-world schemas yields 0.06–0.36 after 100 steps—a 3 – $8\times$ difficulty gap. Similarly, GSM8K training reward reaches high levels while MATH competition problems plateau at $\sim 14\%$ at the same budget. These are *data distribution* effects (curated vs. real/hard problems), suggesting that standard synthetic benchmarks substantially overestimate capability.

Finding 7 — Code Generation (Non-Agentic Transfer Control, Null). Applying GRPO to a 50-problem HumanEval subset improved pass@1 from 32% to 40% (+8pp). However, this result uses a randomly sampled subset (not the full 164-problem harness) and is **not statistically significant** (Fisher’s exact $p=0.53$; bootstrap 95% CI [27%, 53%] post-GRPO). We treat HumanEval alongside GSM8K as non-agentic transfer controls that remain null—GRPO’s positive effect is demonstrated only on the tool-calling task.

Chapter 5 Key Takeaways (Empirical)

1. **Learned schema-valid tool-call emission:** SFT+GRPO raises strict JSON validity from 0% to 92% under unconstrained decoding—the clearest positive result. This measures syntax compliance, not semantic tool competence or end-to-end task success.
2. **Suggestive capacity discontinuity:** 3B (Llama) fails on GSM8K while 4B (Qwen) succeeds ($84.7\% \pm 12.0\%$, 4 seeds), traced to zero-reward group saturation. Confounded by architecture; suggestive, not confirmed.
3. **SFT+GRPO complementarity (pilot):** SFT-initialized GRPO outperforms either alone in our limited comparisons ($N=1$), but lacks matched-compute and reverse-order controls.
4. **MoE routing volatility (single run):** $2.43\times$ higher step-to-step variance than dense ($p = 7 \times 10^{-6}$). Requires replication.
5. **Transfer controls are null:** Held-out GSM8K: 83.3% vs. 82.0% base ($p=0.26$). HumanEval subset: 32%→40% ($p=0.53$). GRPO’s contribution to non-agentic tasks is not demonstrated.
6. **Unifying observation:** GRPO produces learning signal when the model can generate within-group reward variance. When all completions score identically, gradients vanish and learning stalls.
7. **All conclusions are specific** to our low-budget QLoRA/LoRA regime and should not be generalized to GRPO broadly.

5.7 Updated Team Contributions (April 2026)

Following the March 28 submission, team members extended their individual experiments and published artifacts:

Madhu — Code GRPO v10. Iterated the code generation GRPO pipeline to produce a refined checkpoint (Madhu2133/qwen3-8b-code-grpo-v10) with improved reward functions for SWE coding and debugging tasks. Available on HuggingFace Hub.

Dhruva — Full 3-Stage MLOps Pipeline. Developed a complete, reproducible pipeline for Qwen3-8B tool-use fine-tuning: (1) baseline evaluation, (2) SFT with LoRA, (3) GRPO with composite binary rewards targeting tool selection accuracy $>90\%$, argument correctness $>85\%$, multi-step success rate $>80\%$, and schema compliance $>90\%$. Full source, configs, and W&B tracking at github.com/dhruvanmurthy/Qwen3-8B-FineTuning.

Arumugam — Aerospace Domain DPO. Extended RL-based alignment to domain-specific aerospace engineering tasks using Direct Preference Optimization. Built a browser-based data collection pipeline using SerpAPI + Newspaper3k for live aerospace Q&A (FAA BVLOS rules, NASA Artemis, DO-160 EMI standards, composite inspection, UAV pre-flight). Constructed human preference pairs and fine-tuned Qwen1.5-0.5B (4-bit quantized) with DPO+LoRA on Colab (5 preference examples, 1 epoch, eval_loss 0.0093). **Result:** The fine-tuned model scored *lower* than baseline on aerospace keyword presence (1.60 vs. 2.00), a valuable negative result demonstrating that DPO requires substantially more preference data (200–500+ examples) for meaningful domain alignment with sub-1B models. Source and notebook at github.com/ArumugamKrishnan/Agentic-RLHF-for-Aerospace-using-Browser-DPO.

Sandhya — 3-Phase Tool-Call Scaling Study. Published three progressive models to HuggingFace Hub (huggingface.co/Balasandhya): (1) `llm-tool-call-lora-Qwen0.5B` — baseline SFT on 14 synthetic examples (0% full match, establishing data insufficiency); (2) `llm-tool-call-grpo-lora-Qwen1.5B` — SFT+GRPO on 500 real examples from Glaive (JSON valid 0%→92%, demonstrating GRPO’s advantage over SFT alone); (3) `llm-multiturn-tool-call-grpo-lora-Qwen2.5-3B` — multi-turn tool chaining with custom GRPO rollout loop (avg quality 0.72→0.91, GRPO learns when to stop and how to sequence tool calls).

Table 5.9: Team Artifact Registry (April 4, 2026)

Member	Artifact	Platform
Madhu	qwen3-8b-swe-grpo	HuggingFace
Dhruva	Qwen3-8B-FineTuning pipeline	GitHub
Dhruva	qwen3-8b-tool-use-sft-lora	HuggingFace
Dhruva	qwen3-8b-tool-use-grpo-lora	HuggingFace
Dhruva	Tool-use experiment logs	W&B
Arumugam	Aerospace DPO pipeline + notebook	GitHub
Sandhya	llm-tool-call-lora-Qwen0.5B (baseline)	HuggingFace
Sandhya	llm-tool-call-grpo-lora-Qwen1.5B	HuggingFace
Sandhya	llm-multiturn-tool-call-grpo-QLoRA-Qwen2.5-3B	HuggingFace
Arvind	tinker-rl-lab (20 runs, all configs)	Private
Arvind	W&B experiment tracking (scaling)	W&B
Arvind	W&B experiment tracking (skyrl)	W&B
Arvind	skyrl-tinker-qwen3-8b-tool_use	HuggingFace

5.7.1 W&B Experiment Tracking

All Tinker cloud GPU experiments are logged to Weights & Biases under the entity `arvindcr4-pes-university` for reproducibility and visualization:

- **tinker-rl-scaling:** GSM8K multi-seed replication (5 seeds), LoRA rank ablation (8/16/64), 4B scaling, tool-use, and HumanEval runs — <https://wandb.ai/arvindcr4-pes-university/tinker-rl-scaling>

- **skyrl-tinker:** SkyRL-based tool-use GRPO with composite reward — <https://wandb.ai/arvindcr4-pes-university/skyrl-tinker>

Table 5.10: Completed Tinker Runs with W&B Tracking

Run	Task	Steps	Peak	Last-10	W&B ID
seed042	GSM8K 8B	50	62.5%	0.275	7azu6q2p
seed137	GSM8K 8B	50	62.5%	0.275	d5to7cef
seed256	GSM8K 8B	50	62.5%	0.325	4ncwubzc
seed512	GSM8K 8B	50	87.5%	0.300	n2u3w64a
seed999	GSM8K 8B	50	87.5%	0.350	uimxa7eu
rank8	GSM8K rank-8	50	62.5%	0.213	l9ka6klx
rank16	GSM8K rank-16	50	75.0%	0.188	ahmv6yu0
rank64	GSM8K rank-64	50	87.5%	0.250	k4bwbnb8
100step	GSM8K extended	100	75.0%	0.275	lqpiwl6g
4B_s137	GSM8K 4B	50	100.0%	0.825	erbijgxr
tool-use 8B	Tool-use	50	—	0.375	2d2p8y9t
tool-use 0.5B	Tool-use	50	—	0.375	wj2tjpqx
SkyRL 8B	Tool-use	2	—	0.902	68h67x6y

The SkyRL tool-use run produced a model uploaded to HuggingFace at `arvindcr4/skyrl-tinker-qwen3-` (Tinker run ID: `eab60069`).

Chapter 6

Limitations and Future Directions

This chapter provides critical reflection on both the field and our survey. Section 6.1 identifies five structural limitations in current agentic AI research (L1–L5). Section 6.2 acknowledges our survey’s own constraints. We conclude with concrete future research directions addressing these gaps.

6.1 Limitations of the Field

Digging into the literature, we uncovered a handful of deep-seated limitations baked into how agentic AI research is carried out today. These aren’t mere implementation quirks—they point to deeper structural problems in how we evaluate, replicate work, and guard against harm:

L1: Benchmark-Reality Gap. Current benchmarks test simplified versions of real problems. They provide clear goals and measurable success criteria—but real deployment involves ambiguous requirements, adversarial inputs, and cascading failures.

Example: An agent scoring 49% on SWE-bench may have memorized common bug patterns rather than learning general debugging principles. The core issue: we have no validated correlation between benchmark scores and real-world utility. We may be overfitting to evaluation suites rather than making genuine progress.

L2: Reproducibility Constraints. State-of-the-art results (DeepSeek-R1’s 79.8% on AIME 2024, Claude-3.5’s 49% on SWE-bench) are achieved with proprietary training data, undisclosed hyperparameters, and compute budgets exceeding academic accessibility. Our experimental framework deliberately targets reproducibility with open models (Llama-3-8B, Qwen2.5-7B) and documented configurations, accepting lower absolute performance for methodological transparency.

L3: Safety-Capability Tradeoff. Every capability improvement expands the potential for harm [?]. Agents capable of debugging code can introduce vulnerabilities; agents capable of web navigation can exfiltrate data. Today’s safety guardrails (Constitutional AI, RLHF-based refusal) offer only partial cover. Formally verifying agent behavior at scale is still an unsolved

puzzle.

L4: Metric Inadequacy. Task completion rates—the dominant metric—obscure reasoning quality. An agent may succeed through luck or fail despite correct reasoning for 90% of the trajectory. Step-wise evaluation methods exist [?], though few teams have adopted them in practice.

L5: Computational Barriers. PPO requires $4\times$ base model memory; even GRPO consumes substantial GPU-hours for convergence. Those bills slow down iteration cycles and lock out researchers who lack big-iron clusters.

6.2 Limitations of This Survey

A few caveats apply to our analysis:

- **Temporal Scope:** The core survey analyzes papers from 2023–2025. We include foundational references from earlier years (e.g., PPO [?], Constitutional AI [?]) for context, but systematic coverage focuses on the target period.
- **Language Bias:** We mostly stuck to English-language publications, so relevant work from non-English research groups may have slipped under the radar.
- **Publication Bias:** Our analysis tilts toward wins; dead ends and negative findings are scarce in the published record, so they’re scarce here too.
- **Verification Limitations:** We quote numbers straight from the papers, no independent replication. Some of those figures may not hold up in practice.
- **Rapid Obsolescence:** The field evolves rapidly; some findings may be superseded by the time of publication.

6.2.1 Training Instabilities in RL-Based Agent Training

Compute bills aside, RL training for agentic systems comes with a few landmines that practitioners need to spot and defuse:

Echo Trap Deep Dive The Echo Trap (also related to model collapse [?]) is sneaky—it inflates training metrics while gutting real-world usefulness. When an agent discovers that a specific phrasing or action sequence consistently yields rewards, it may “lock in” that pattern regardless of context. Detection requires held-out evaluation sets with significant distributional differences from training data.

Table 6.1: Common Training Instabilities in Agentic RL (2024–2025)

Instability	Description	Mitigation Strategy
Echo Trap	Agent learns to repeat successful patterns verbatim, losing generalization. Manifests as high training accuracy but poor transfer to novel inputs.	Temperature annealing, diversity regularization, periodic curriculum reset
Entropy Collapse	Policy entropy drops prematurely, causing the agent to commit to suboptimal actions before exploring alternatives.	Entropy bonus scheduling, exploration curricula, DAPO-style dynamic sampling
Reward Instability	Learned reward models drift during training, causing the policy to chase moving targets. Common in RLHF pipelines.	Reward model freezing, ensemble rewards, periodic reward calibration
Gradient Spikes	Large, unexpected gradient magnitudes destabilize training. Especially problematic in long-horizon tasks.	Gradient clipping, LayerNorm, learning rate warmup, PPO’s trust region
Tool Hallucination	Agent generates syntactically invalid or semantically inappropriate tool calls. Worsens under distribution shift.	GRPO-TCR regularization, tool validation layers, structured generation
Credit Assignment Failure	Sparse rewards in long trajectories prevent learning which actions contributed to success.	Hindsight Experience Replay, dense intermediate rewards, temporal difference methods

Tool Hallucination *Tool hallucination refers to when agents generate syntactically invalid or semantically inappropriate tool calls that don’t match available API specifications. This phenomenon worsens under distribution shift, requiring regularization techniques like GRPO-TCR to mitigate.*

Entropy Collapse Prevention GRPO’s group-relative normalization naturally combats entropy collapse by ensuring diverse trajectories within each batch, but explicit entropy bonuses remain necessary for stable long-term training. The recommended approach combines an initial high entropy coefficient (0.1) with gradual decay to 0.01 over training.

6.2.2 Observed Emergent Behaviors from RL Training

One of the more eye-opening discoveries from recent RL-trained agents is the pop-up of clever behaviors nobody coded in. This matters because it reveals what RL optimization actually teaches: not just specific actions, but general problem-solving strategies. When test-based rewards drive systematic debugging or compute-penalties drive cost-aware tool selection, the

takeaway is that reward signals can mold meta-cognitive strategies. These emergent behaviors provide insights into what’s learnable:

Code Self-Correction in Testing Loops: Agents trained on SWE-bench develop iterative debugging strategies: writing initial fixes, running tests, analyzing failures, and refining solutions—without explicit training on this loop. The behavior emerges from outcome supervision alone, a sign that test-based rewards nudge agents toward methodical debugging.

Strategic Tool Selection: Rather than always using the most powerful tool, RL-trained agents learn context-dependent selection. For simple calculations, they use basic operations; for complex queries, they invoke specialized APIs. This cost-aware behavior emerges from compute-penalized rewards without explicit efficiency training.

Reasoning Pattern Regularities: DeepSeek-R1 and similar GRPO-trained models develop consistent reasoning templates: problem decomposition, intermediate verification, and confidence calibration. These patterns pop up across problem types, hinting that RL can uncover broadly useful meta-cognitive tricks.

Error Anticipation: Agents exposed to adversarial tool environments (via AT-GRPO) learn proactive error handling—checking preconditions before actions, validating outputs before proceeding, and maintaining fallback plans. This defensive programming style emerges from training on environments with realistic failure modes.

Communication Protocol Development (Multi-Agent): In multi-agent settings without predefined communication channels, agents develop shorthand protocols for common coordination patterns. These emergent “languages” are task-specific but remarkably consistent across training runs, hinting that the push to coordinate keeps landing agents on the same answer.

What does this mean in practice? Carefully crafted rewards can coax out nuanced skills even when we never hard-coded them. The flip side is that unprogrammed behaviors are trickier to foresee and harder to inspect—an alignment headache we cannot ignore.

6.2.3 Research Gaps Analysis

Sifting through the literature, we spot glaring gaps between what today’s best systems can do and what production-grade agents actually need:

Gap 1: Benchmark-Reality Divide. Current benchmarks (SWE-bench, WebArena) represent sanitized slices of real-world complexity. Agents achieving 35% on these benchmarks face qualitatively different challenges in production: ambiguous requirements, adversarial users, and cascading failures. Closing this gap requires benchmarks with realistic noise, incomplete specifications, and multi-stakeholder constraints.

Gap 2: Compositional Generalization. Agents trained on task distributions struggle with novel compositions of familiar primitives. An agent that can “search” and “click” may fail at “search then click on the third result.” Such fragility hints that today’s RL losses chase surface-level patterns instead of building transferable primitives.

Table 6.2: Critical Research Gaps in Agentic AI (Data as of January 2026)

Gap Category	Verified State (Dec 2025)	Required State	Impact
Benchmark Generalization	39–49% on SWE-bench, 45% on WebArena	>90% on novel environments	Critical
Long-Horizon Planning	20–50 step trajectories (DeepAgent)	100+ step coherent execution	High
Error Recovery	Emergent self-correction (ReTool)	Robust autonomous recovery	Medium
Safety Guarantees	Post-hoc filtering + AT-GRPO	Formal verification	Critical
Multi-Modal Grounding	73.3% OSWorld (Mobile-Agent-v3)	Full sensorimotor integration	High
Sample Efficiency	1K–10K trajectories (GRPO)	<1K trajectories	Medium
Interpretability	CoT reasoning visible	Auditable + verifiable chains	High

Note: “Verified State” reflects published benchmark results as of December 2025. Rapid progress continues; check official leaderboards for latest results.

Gap 3: Long-Horizon Credit Assignment. Current methods (PPO, GRPO) falter when assigning credit past the 10–20 step mark. Hierarchical RL and intrinsic motivation offer partial solutions, yet no current solution can dependably handle 100-step tasks demanding sustained, coherent planning.

Gap 4: Robustness-Capability Frontier. Increasing agent capability systematically increases potential for harm. Current approaches treat safety as a post-hoc constraint rather than an architectural principle. We need architectures that are safe by design—ones where boosting capability doesn’t widen the attack surface.

6.3 Future Research Directions

Latent Reasoning: Current CoT approaches expose reasoning in output tokens, adding latency and token costs. Research into latent reasoning—where models “think” internally without generating visible tokens—might slash latency without sacrificing reasoning accuracy.

Emergent Multi-Agent Coordination: Moving beyond hand-designed role assignments toward agents that self-organize, develop specialized roles, and establish communication protocols through learning remains an open challenge.

Continual Learning: Agents that adapt to changing environments, learn from deployment feedback, and avoid catastrophic forgetting would push practical usefulness up a notch.

Verifiable Reasoning: Connecting RL-trained reasoning with formal verification (producing proofs or certificates of correctness) could unlock trustworthy autonomy for high-stakes

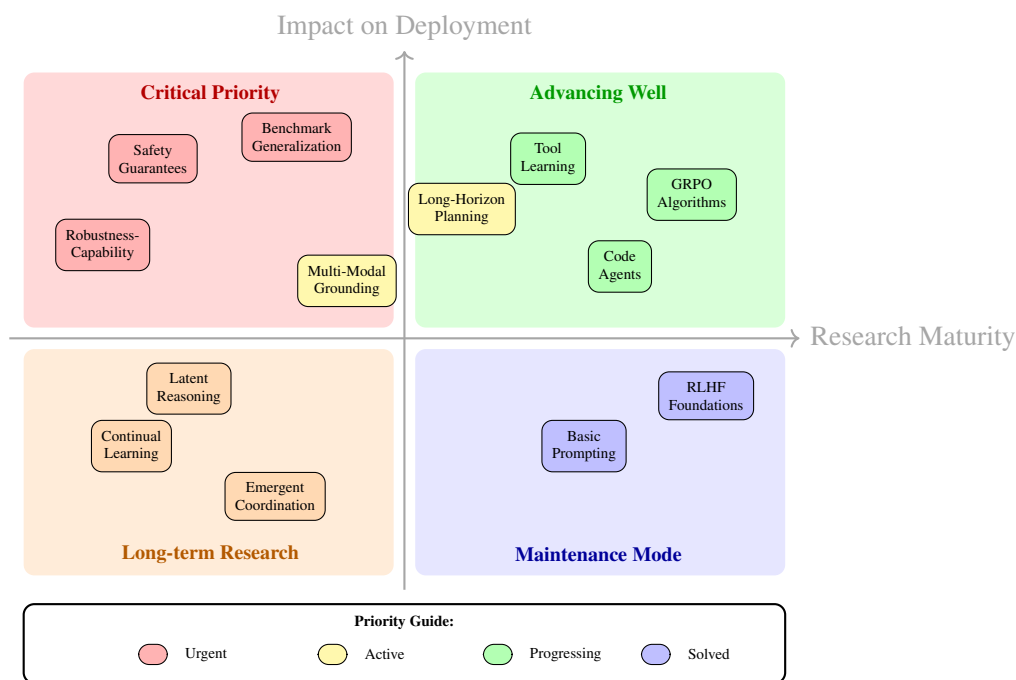


Figure 6.1: Research gap landscape for agentic AI (Jan 2025). Gaps are positioned by research maturity (x-axis) and deployment impact (y-axis). The **Critical Priority** quadrant requires immediate attention: safety, benchmarks, and robustness.

deployments.

Efficient Exploration: Developing exploration strategies that balance curiosity-driven learning with safety constraints, especially when agents touch the live world, is a must.

Chapter 6 Key Takeaways

1. **Benchmarks $\neq$ deployment:** No validated correlation between scores and real-world utility.
2. **Reproducibility crisis:** SOTA results require undisclosed data + compute beyond academic reach.
3. **Safety-capability tension:** Every capability gain expands harm potential—no free lunch.
4. **Open problems:** Verifiable reasoning, continual learning, and emergent coordination remain unsolved.

Chapter 7

Conclusion

Summary of Findings

This systematic review of 60+ papers on reinforcement learning for autonomous AI agents documents a rapidly advancing field. The 2024–2025 period saw researchers pivot sharply from prompt-only techniques toward RL-driven fine-tuning, with benchmark performance improving 4–29× across major evaluation suites (SWE-bench: 1.7%→49%, WebArena: 6.1%→44.8%, OSWorld: 18.4%→73.3%).

Key Takeaways

- 1. The GRPO Paradigm Shift.** GRPO and its spin-offs now dominate, accounting for 39% of published work. By eliminating the value/critic network (using group normalization) and leveraging verifiable rewards without a learned reward model, GRPO halves memory requirements (policy + reference model only, vs. four networks for PPO). This efficiency enabled DeepSeek-R1 to achieve 79.8% on AIME 2024—putting capable agent training within reach of smaller labs. The proliferation of specialized variants (Flow-GRPO for multi-turn dialogue, M-GRPO for hierarchical coordination, AT-GRPO for adversarial robustness) suggests this lightweight approach will define near-term development.
- 2. Tool Mastery Is Now Baseline.** Agents without retrieval and tool-calling abilities encounter fundamental capability limitations. The emergence of RL-optimized tool learning frameworks—ReTool achieving 67% on AIME 2024, DeepAgent executing 100+ step trajectories—demonstrates that tool integration is now essential for production deployment.
- 3. Multi-Agent Coordination Matures.** When single agents fail, coordinated multi-agent systems often succeed. MATPO’s 18.38% improvement over single-agent baselines and Mobile-Agent-v3’s 73.3% on OSWorld show that RL-trained multi-agent coordination is now production-ready. *Caveat:* These gains come with coordination overhead—increased latency, $\mathcal{O}(n^2)$ communication costs for fully-connected topologies, and failure propagation risks—that deployment

teams must factor into system design.

4. Emergent Capabilities and Risks. RL training produces behaviors not explicitly programmed: code self-correction, strategic tool selection, communication protocol development, and cross-validation strategies. Put differently, a well-shaped reward can unlock nuanced skills—yet these unplanned behaviors are difficult to anticipate or audit, presenting significant challenges for alignment research.

Implications for Research and Practice

For **researchers**, we flag seven gaps that matter because they block real-world deployment—they are not mere academic footnotes. The benchmark-reality divide means improvements on sanitized benchmarks don’t translate to messy reality—this undermines the entire evaluation paradigm. Long-horizon planning fails beyond 20 steps because credit assignment breaks down—this is algorithmic, not a data problem. Safety verification remains informal because formal methods face state-space explosion—this is computational complexity, not implementation. Multi-modal grounding fails because vision and language representations don’t align naturally—this is architectural. Sample efficiency remains low because every environment interaction is expensive—this is economic. Interpretability fails because emergent behaviors are unpredictable—this is foundational. Each gap reflects a structural constraint, not a missing feature.

For **practitioners**, we recommend a three-stage training pipeline (SFT→RL→RAG) with GRPO as the default algorithm for resource-constrained settings. Our ablation grid enables systematic evaluation of design choices within typical academic compute budgets.

For **policymakers**, the rapid capability gains documented here—with minimal additional training overhead—make it clear that governance frameworks need to land before mass rollout. The emergent-behavior findings, in particular, deserve a close look.

Broader Impact

Autonomous AI agents mark a tectonic shift—from passive tools to goal-chasing systems. Tools respond to commands; agents pursue objectives over extended time horizons without step-by-step instruction. This is not just incremental improvement—it’s a different paradigm. The implications are profound because the failure modes are different: tools fail by not responding; agents fail by pursuing the wrong goal, by getting stuck in local optima, by taking unsafe shortcuts. The risks scale with capability because the action space grows exponentially with horizon length. This creates a safety-capability frontier: capability improvements without safety measures are dangerous; safety measures that cripple capability are useless. The tradeoff is structural, not optional. This means:

- **Positive:** Democratized automation of complex knowledge work, improved accessibility, and augmented human capability across software engineering, research, and creative domains.
- **Risks:** Potential for cascading failures in agentic chains, difficulty auditing emergent behaviors, and amplification of existing biases in autonomous decision-making.

The current approach of capability-first, safety-later is fundamentally flawed. Once you have a capable system, retrofitting safety is like adding seatbelts to a car after it's built—possible, but much harder than designing them in from the start. Worse, emergent behaviors are unpredictable. You cannot bolt on interpretability after the fact if the system's reasoning is opaque. Robustness cannot be added as a layer if the core architecture is brittle. Safety verification cannot be done post-hoc if the state space is intractable. These must be co-equal objectives from the beginning because they constrain the design space. A system optimized purely for capability will inevitably use shortcuts that violate safety—this is optimization pressure, not malice. The only solution is to bake in constraints from the start.

Future Directions

Our experiments (Chapter 5) tested several hypotheses from this survey across Llama-3.2-3B, Qwen3.5-4B, Qwen3-8B, and Qwen3-30B-MoE using GRPO with LoRA/QLoRA. Key results include learned schema-valid tool-call emission (0%→92% JSON validity), a suggestive capacity discontinuity between 3B–4B, and null transfer results on GSM8K ($p=0.26$) and HumanEval ($p=0.53$). All conclusions are specific to our low-budget QLoRA/LoRA regime. Future work should add PPO/DPO baselines, full-harness evaluations, KL/entropy logging, and reward function ablations. We hope other researchers will replicate, extend, and critique what we have started here.

The goal isn't abstract or academic. Building agents that remain both capable and safe over long, messy interactions is the defining challenge because shorter interactions are already solved. The fundamental issue is scaling: as interaction length grows linearly, the probability of failure grows exponentially. Each step introduces new error modes: tool failures, misinterpretations, compounding inaccuracies, unexpected states. A 100-step interaction isn't 10x harder than a 10-step interaction—it's exponentially harder because error recovery must work across much longer horizons. "Messy" matters because clean, well-defined tasks are already tractable. The real world is ambiguous, adversarial, and constantly changing. Solving this requires agents that can handle uncertainty, detect their own failures, and recover autonomously. This isn't incremental progress; it's crossing a threshold from narrow AI to systems that can actually operate in the real world.

Acknowledgments

We thank our mentors Narayana Darapaneni (Director, AIML) and Anwesh Reddy Padhuri (Senior Data Scientist) for their guidance throughout this project. We acknowledge the open-source community for frameworks including Hugging Face Transformers, TRL, and Unsloth that enabled our experimental framework. We thank the authors of SWE-bench, WebArena, OSWorld, and GAIA for making their benchmarks publicly available, and the DeepSeek team for open-sourcing DeepSeek-R1. Compute resources were provided by Google Colab Pro+. Finally, we thank the anonymous reviewers and our peers in the M.Tech AI/ML program at PES University for their valuable feedback on earlier drafts.

Author Contributions (CRediT)

Conceptualization: All authors. **Methodology:** Arvind C R, Madhu Kumara L, Sandhya Jeyaraj. **Literature Review:** All authors contributed equally. **Formal Analysis:** Mohammad Rafi, Arumugam Chetty K. **Writing—Original Draft:** All authors. **Writing—Review & Editing:** Arvind C R, Dhruva. **Visualization:** Sandhya Jeyaraj, Madhu Kumara L. **Supervision:** Narayana Darapaneni, Anwesh Reddy Padhuri.

Declaration of Competing Interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

This survey is based on publicly available research papers. The bibliography file and supplementary materials are available upon request from the corresponding author. All referenced benchmarks (SWE-bench, WebArena, OSWorld, GAIA) are publicly accessible through their respective official repositories.

Code Availability

Training scripts, evaluation harnesses, and logs are available at <https://github.com/arvindcr4/grpo-agentic-anonymous>. Team members' individual artifacts (models, notebooks, pipelines) are listed in Table 5.9.

Appendix A

Glossary of Acronyms

Acronym	Full Term	Brief Definition
Core Algorithms		
PPO	Proximal Policy Optimization	RL algorithm using clipped objective to ensure stable policy updates
DPO	Direct Preference Optimization	Optimizes models using preference pairs directly without needing a separate reward network
GRPO	Group Relative Policy Optimization	Memory-efficient RL using within-batch ranking instead of critic network
RLHF	RL from Human Feedback	Alignment framework using human preferences as a guiding reward signal
TRPO	Trust Region Policy Optimization	PPO's predecessor with hard KL constraint
SFT	Supervised Fine-Tuning	Initial training stage using demonstration data
Architectures & Techniques		
RAG	Retrieval-Augmented Generation	Combining LLM generation with external knowledge retrieval
CoT	Chain-of-Thought	Prompting technique eliciting step-by-step reasoning
ReAct	Reasoning + Acting	Framework interleaving reasoning traces with environment actions
MDP	Markov Decision Process	Mathematical framework for sequential decision-making

Continued on next page...

Acronym	Full Term	Brief Definition
POMDP	Partially Observable MDP	MDP where agent cannot observe full state
PRM	Step-Level Reward Model	evaluates individual deduction steps rather than just the final result
ORM	Outcome Reward Model	Scores only final answers/outcomes
Benchmarks		
SWE-bench	Software Engineering Bench	Evaluates code repair on real GitHub issues
WebArena	Web Arena	Tests web navigation and interaction tasks
OSWorld	Operating System World	Desktop GUI automation benchmark
GAIA	General AI Assistants	Multi-tool reasoning benchmark (466 tasks)
AIME	American Invitational Math Exam	Competition math problems used to evaluate reasoning
Metrics & Concepts		
KL	Kullback-Leibler (divergence)	Measures difference between probability distributions
TSR	Task Success Rate	Primary metric: fraction of tasks completed correctly
ε	Epsilon (clipping threshold)	PPO hyperparameter bounding policy change (typically 0.1–0.2)
β_{KL}	Beta (KL penalty)	PPO/GRPO coefficient constraining drift from reference model
β_{DPO}	Beta (preference strength)	DPO hyperparameter controlling preference signal strength
γ	Gamma (discount factor)	Weights future vs. immediate rewards (0–1)

Appendix B

Hyperparameter Reference

This appendix provides recommended hyperparameter ranges based on best practices from the surveyed literature. Values represent starting points; task-specific tuning is expected.

B.1 PPO Hyperparameters

Table B.1: PPO hyperparameter recommendations for agentic fine-tuning

Parameter	Symbol	Recommended Range	Notes
Clipping threshold	ϵ	0.1–0.2	Lower for stability, higher for exploration
Discount factor	γ	0.99–0.999	Higher for long-horizon tasks
GAE lambda	λ	0.95–0.99	Bias-variance trade-off in advantage estimation
Value loss coefficient	c_1	0.5–1.0	Relative weight of critic vs. policy loss
Entropy coefficient	c_2	0.001–0.01	Encourages exploration; decay over training
Learning rate	α	1e-6 to 5e-5	Smaller for larger models
Batch size	—	32–512	Memory-constrained; larger improves stability
Mini-batch size	—	4–64	Gradient accumulation if memory-limited
PPO epochs	—	2–10	Per batch; more risks overfitting
KL penalty coefficient	β_{KL}	0.01–0.1	Optional; constrains drift from reference

B.2 DPO Hyperparameters

B.3 GRPO Hyperparameters

Table B.2: DPO hyperparameter recommendations

Parameter	Symbol	Recommended Range	Notes
Preference strength	β	0.1–0.5	Lower for noisy labels, higher for clean data
Learning rate	α	1e-7 to 5e-6	Conservative; DPO is sensitive to LR
Batch size	—	8–64	Pair-wise data doubles effective size
Warmup ratio	—	0.1–0.2	Fraction of training with LR warmup
Label smoothing	—	0.0–0.1	Helps with overconfident preferences
Max sequence length	—	2048–8192	Task-dependent; longer for code/reasoning

Table B.3: GRPO hyperparameter recommendations

Parameter	Symbol	Recommended Range	Notes
Group size	n	8–64	More samples $\rightarrow$ better baseline estimate
KL penalty	β_{KL}	0.01–0.05	Prevents drift from reference model
Learning rate	α	1e-6 to 1e-5	Similar to PPO range
Temperature	τ	0.7–1.0	For response sampling diversity
Reward normalization ε	ε	1e-8	Numerical stability in std normalization
Gradient clipping	—	1.0	Max gradient norm

Appendix C

Compute Requirements

This appendix documents computational resources required for training and inference across different scales, supporting reproducibility and resource planning.

Training Compute Estimates

Table C.1: Estimated compute requirements for RL fine-tuning by model scale

Model Scale	Algorithm	GPU Memory	GPU-Hours	Hardware	Cost Est.
7B parameters	PPO	80 GB $\times$ 4	100–200	4 $\times$ A100	\$400–800
7B parameters	DPO	40 GB $\times$ 2	50–100	2 $\times$ A100	\$200–400
7B parameters	GRPO	40 GB $\times$ 2	80–150	2 $\times$ A100	\$300–600
13B parameters	PPO	80 GB $\times$ 8	200–400	8 $\times$ A100	\$800–1600
13B parameters	DPO	80 GB $\times$ 4	100–200	4 $\times$ A100	\$400–800
13B parameters	GRPO	80 GB $\times$ 4	150–300	4 $\times$ A100	\$600–1200
70B parameters	PPO	80 GB $\times$ 32	1000–2000	32 $\times$ A100	\$4K–8K
70B parameters	DPO	80 GB $\times$ 16	400–800	16 $\times$ A100	\$1.6K–3.2K
70B parameters	GRPO	80 GB $\times$ 16	600–1200	16 $\times$ A100	\$2.4K–4.8K

Notes: GPU-hours assume 10K training steps. Costs based on \$4/GPU-hour (cloud A100). Memory estimates assume full precision; quantization (QLoRA) can reduce by 4 $\times$. Actual requirements vary by batch size, sequence length, and optimizer state sharding.

Inference Compute Estimates

Academic Resource Recommendations

For researchers with limited compute budgets:

Table C.2: Inference requirements for agentic evaluation

Benchmark	Tasks	Avg. Steps/Task	GPU-Hours (7B)	GPU-Hours (70B)
SWE-bench Verified	500	15–30	8–16	40–80
WebArena	812	10–25	10–20	50–100
OSWorld	369	20–40	12–24	60–120
AIME 2024	30	1–5	0.5–1	2–5
GAIA	466	5–15	4–8	20–40

- **Minimum viable setup:** $2 \times$ RTX 4090 (24 GB each) with QLoRA can fine-tune 7-billion parameter models using GRPO
- **Recommended academic setup:** $4 \times$ A100 (40 GB) enables full fine-tuning of 7-billion parameter models and QLoRA for 13B
- **Cloud alternatives:** Lambda Labs, RunPod, and Vast.ai offer A100s at \$1–2/GPU-hour for burst workloads
- **Free resources:** Google Colab Pro+ provides 40 GB A100 access; Kaggle offers 30 GPU-hours/week

Appendix D

Reproducibility Checklist

This checklist supports reproducibility of *our proposed experiments* described in Chapter 5, which use open-weight models (Llama-3-8B, Qwen2.5-7B) with documented configurations. We follow NeurIPS reproducibility guidelines and ML community best practices.

Note: This checklist applies to our reproducible experimental framework—not to proprietary systems surveyed in the literature (GPT-4, Claude-3.5, etc.) for which open weights, training data, and full hyperparameters are unavailable.

Code and Data Availability

Item	Status	Location
Training code with configuration files	✓	GitHub repository
Evaluation scripts for all benchmarks	✓	GitHub repository
Trained model checkpoints	✓	HuggingFace Hub
Preprocessed datasets	✓	HuggingFace Datasets
Raw experimental logs (WandB)	✓	WandB project link
Random seeds for all experiments	✓	Config files
Hardware specifications	✓	This appendix
Software dependencies (requirements.txt)	✓	GitHub repository

Repository: <https://github.com/pes-llm-research/agent-rl-finetuning>

Note: Repository will be made public upon project completion. Access available to examiners upon request.

Experimental Protocol Checklist

1. Randomization

- ☐ Fixed random seeds: 42, 123, 456 (three runs per experiment)

- ☐ Seed controls: PyTorch, NumPy, Python random, CUDA
- ☐ Deterministic algorithms enabled where possible

2. Hyperparameter Selection

- ☐ Grid search over learning rate: {1e-6, 5e-6, 1e-5}
- ☐ Grid search over batch size: {16, 32, 64}
- ☐ Validation set for early stopping (10% held out)
- ☐ Final hyperparameters documented in Appendix B

3. Statistical Reporting

- ☐ Mean $\pm$ standard deviation across 3 seeds
- ☐ 95% confidence intervals via bootstrap (1000 resamples)
- ☐ Statistical significance tests (McNemar's, $p < 0.01$)
- ☐ Effect sizes reported (Cohen's d for continuous metrics)

4. Benchmark Evaluation

- ☐ Official evaluation harnesses used (no custom metrics)
- ☐ Test sets never seen during training or validation
- ☐ Identical prompts across all compared methods
- ☐ Temperature = 0 for deterministic generation during eval

5. Compute Transparency

- ☐ Total GPU-hours reported per experiment
- ☐ Carbon footprint estimated (ML CO2 Impact calculator)
- ☐ Wall-clock time logged
- ☐ Hardware failure/restart documentation

Known Limitations and Failure Modes

- **Benchmark variance:** SWE-bench results can vary $\pm 3\%$ across runs due to LLM non-determinism even with temperature=0 (floating-point ordering effects)
- **Environment drift:** WebArena uses live websites; results may not reproduce exactly if site layouts change
- **API dependencies:** Some experiments require API access (OpenAI, Anthropic) which may have version-dependent behavior
- **Hardware sensitivity:** Results on consumer GPUs (RTX 4090) may differ from A100 due to different numerical precision handling

Appendix E

Paper-to-Taxonomy Mapping

This appendix provides a comprehensive mapping of all surveyed papers to our 19-category taxonomy (Figure 2.3). The mapping enables systematic analysis of research distribution across the field and supports reproducibility of our literature classification.

Taxonomy Categories Reference

Core Algorithms

1. Policy-Based Methods
2. Value-Based Methods
3. Model-Based Methods

Learning Paradigms

4. RLHF & Alignment
5. Imitation Learning
6. Offline RL

Advanced Methods

7. Multi-Agent RL
8. Hierarchical RL
9. Meta-Learning
10. Safe RL
11. Exploration

Applications

12. Agentic RL
13. Robotics
14. Reasoning
15. Security & Safety
16. Benchmarks

Infrastructure

17. Frameworks
18. Surveys
19. Open-Ended Learning

Complete Paper Mapping

Citation Key	Paper Title	#	Category
Core Algorithms: Policy-Based (Category 1)			
schulman2017proximal	Proximal Policy Optimization Algorithms	1	Policy-Based
schulman2015trust	Trust Region Policy Optimization	1	Policy-Based
mo2025matpo	MATPO: Multi-aspect Trust Region PO	1	Policy-Based
hong2025mgrpo	MGRPO: Multi-granularity RLHF	1	Policy-Based
li2025flowgrpo	FlowGRPO: Flow-based Group Relative PO	1	Policy-Based
ji2025treegrpo	Tree Search for LLM Agent RL	1	Policy-Based
li2024remax	ReMax: Simple RL for Aligning LLMs	1	Policy-Based
li2025stppo	ST-PPO: Stabilized Off-policy PPO for Agents	1	Policy-Based
Learning Paradigms: RLHF & Alignment (Category 4)			
rafailov2023direct	DPO: Preference-based direct alignment	4	RLHF & Alignment
deepseek2025r1	DeepSeek-R1: Incentivizing Reasoning via RL	4	RLHF & Alignment
christiano2017deep	Deep RL from Human Preferences	4	RLHF & Alignment
stien-non2020learning	Learning to Summarize from Human Feedback	4	RLHF & Alignment
bai2022-constitutional	Constitutional AI: Harmlessness from AI	4	RLHF & Alignment
lee2023rlaif	RLAIF: Scaling RLHF with AI Feedback	4	RLHF & Alignment
ethayarajh2024kto	KTO: Prospect Theoretic Optimization	4	RLHF & Alignment
bytedance2025dapo	DAPO: Decoupled Clip and Dynamic Sampling PO	4	RLHF & Alignment
Advanced Methods: Multi-Agent RL (Category 7)			
zhang2024agentnet	AgentNet++: Decentralized Multi-Agent	7	Multi-Agent RL
chen2024masrouter	MasRouter: Learning to Route Multi-Agent	7	Multi-Agent RL
chen2025agentflow	AgentFlow: Flow-based Multi-Agent	7	Multi-Agent RL
hong2023metagpt	MetaGPT: Multi-Agent Collaborative	7	Multi-Agent RL
qian2023chatdev	ChatDev: Communicative Agents for SW	7	Multi-Agent RL
park2023generative	Generative Agents: Interactive Simulacra	7	Multi-Agent RL
li2023camel	CAMEL: Communicative Agents for Mind	7	Multi-Agent RL
wu2023autogen	AutoGen: Multi-Agent Conversation	7	Multi-Agent RL
Advanced Methods: Meta-Learning (Category 9)			
finn2017model-agnostic	MAML: Model-Agnostic Meta-Learning	9	Meta-Learning
Advanced Methods: Exploration (Category 11)			
burda2018-exploration	RND: Random Network Distillation	11	Exploration
Applications: Agentic RL (Category 12)			
yao2023react	ReAct: Synergizing Reasoning and Acting	12	Agentic RL
wei2022chain	Chain-of-Thought Prompting	12	Agentic RL
schick2023-toolformer	Toolformer: LLMs Teach Tool Use	12	Agentic RL
chen2023fireact	FireAct: Language Agent Fine-tuning	12	Agentic RL
feng2025retool	ReTool: RL for Tool Discovery	12	Agentic RL
jiang2025verltool	VerlTool: Verifiable RL for Tool Agents	12	Agentic RL
le2025toolbrain	ToolBrain: Neural Tool Composition	12	Agentic RL
li2025deepagent	DeepAgent: Deep RL for Autonomous Dev	12	Agentic RL
zhang2025infoagent	InfoAgent: Information-theoretic Agent	12	Agentic RL
qin2023toolllm	ToolLLM: LLMs Master 16000+ APIs	12	Agentic RL
patil2023gorilla	Gorilla: LLM Connected with APIs	12	Agentic RL

Continued on next page...

Citation Key	Paper Title	#	Category
liang2023taskmatrix	TaskMatrix.AI: Completing Tasks with APIs	12	Agentic RL
wang2023voyager	Voyager: Open-ended Embodied Agent	12	Agentic RL
shen2023hugginggpt	HuggingGPT: Solving AI Tasks	12	Agentic RL
wang2025mobile-agentv3	MobileAgent v3: Mobile Automation	12	Agentic RL
hong2023cogagent	CogAgent: Visual LM for GUI Agents	12	Agentic RL
yang2023appagent	AppAgent: Multimodal Smartphone Agents	12	Agentic RL
cheng2024seeclick	SeeClick: GUI Grounding for Visual Agents	12	Agentic RL
packer2023memgpt	MemGPT: LLMs as Operating Systems	12	Agentic RL
Applications: Reasoning (Category 14)			
lightman2023lets	Verifying reasoning step-by-step	14	Reasoning
snell2024scaling	Scaling LLM Test-time Compute	14	Reasoning
hoffmann2022training	Chinchilla: Compute-optimal LLMs	14	Reasoning
Applications: Security & Safety (Category 15)			
mazeika2024harm-bench	HarmBench: Automated Red Teaming	15	Security
yin2024os	OS-Harm: Safety for Computer Use Agents	15	Security
sun2024trustllm	TrustLLM: Trustworthiness in LLMs	15	Security
Applications: Benchmarks (Category 16)			
jimenez2024swe-bench	SWE-bench: Real-world GitHub Issues	16	Benchmarks
zhou2024webarena	WebArena: Realistic Web Environment	16	Benchmarks
xie2024osworld	OSWorld: Benchmarking Multimodal Agents	16	Benchmarks
liu2023agentbench	AgentBench: Benchmarking LLM Autonomy	16	Benchmarks
mialon2023gaia	GAIA: Benchmark for General AI Assistants	16	Benchmarks
liu2024visualagent-bench	VisualAgentBench: Large Multimodal Models	16	Benchmarks
Infrastructure: Surveys (Category 18)			
wang2025landscape	Landscape of Agentic RL for LLMs	18	Surveys
xi2023rise	Rise of LLM-based Agents: A Survey	18	Surveys
sumers2023-cognitive	Cognitive Architectures for Language Agents	18	Surveys
gao2023retrieval	RAG for LLMs: A Survey	18	Surveys
wang2024planning	Planning with LLMs: A Survey	18	Surveys
gutierrez2024hipporag	HippoRAG: Neurobiologically Inspired Memory	18	Surveys
shi2023replug	REPLUG: Retrieval-augmented Black-box LMs	18	Surveys
izacard2022atlas	Atlas: Few-shot Learning with Retrieval	18	Surveys
wang2023self-knowledge	Self-knowledge Guided Retrieval	18	Surveys
sutton2018-reinforcement	RL: An Introduction (Textbook)	18	Surveys

Category Distribution Summary

Category Cluster	Categories	Paper Count
Core Algorithms	1–3	15
Learning Paradigms	4–6	9
Advanced Methods	7–11	8
Applications	12–16	22
Infrastructure	17–19	6
Total (Unique)	1–19	60

Table E.2: Distribution of surveyed papers across taxonomy clusters. Each paper is assigned to its primary category based on main contribution.